Build Agent Quality Upgrade

Build Agent Quality Upgrade

Official Update Outline

1. Update Title

Build Agent Quality Upgrade
Primary Goal: produce higher-quality build kits with stronger extraction, planning, validation, traceability, and verification.

2. Purpose

The Build Agent currently needs a stronger path from source inputs to final kit output. This update is meant to prevent shallow builds, incomplete repo output, weak traceability, and kits that package successfully even when important implementation obligations were missed.

3. Core Outcome

After this update, the Build Agent should:

- consume the full build authority from the kit, not a shallow summary
- derive a justified repo plan before generation starts
- generate only warranted artifacts
- prove requirement coverage
- fail fast on incomplete or low-confidence builds
- package only verified kits

---

4. Upgrade Objectives

BAQ-01 — Full Kit Consumption

The Build Agent must extract and use all required layers of the build kit before generation begins.

Target result

- no major kit section is silently ignored

- every required layer is marked as consumed, deferred, or not applicable with reason

BAQ-02 — Pre-Build Planning

The Build Agent must create a complete derived build plan before file generation.

Target result

- output structure is planned first
- generation follows plan instead of improvising file structure

BAQ-03 — Build Coverage Integrity

Every requirement, unit, and acceptance item must map to implementation targets, proof targets, or explicit non-applicable rulings.

Target result

- no invisible requirement loss
- missing implementation obligations are caught early

BAQ-04 — Minimum Sufficient Output

The Build Agent should generate the minimum artifact set required for a correct, credible build.

Target result

- less bloat
- fewer fake or placeholder files
- stronger output quality per file

BAQ-05 — Verification-Backed Packaging

A build kit should not package as complete unless extraction, planning, generation, and proof checks all pass.

Target result

- packaging becomes a trust boundary, not just a zip boundary

---

5. Scope of the Update

In scope

- build kit extraction rules
- derived planning pack
- expected repo inventory generation
- requirement traceability
- generation sufficiency policy
- validation gate upgrades
- build failure classification
- packaging integrity rules
- test and verification upgrades

Out of scope

- redesigning the entire Axion pipeline
- changing unrelated control plane responsibilities
- replacing build templates wholesale
- UI facelift work unless directly needed for build diagnostics

---

6. New Functional Blocks

BAQ-06 — Kit Extraction Contract

Add a formal extraction stage that records what the Build Agent actually consumed.

Must extract from kit

- brief and mission
- canonical spec
- work breakdown
- acceptance map
- architecture obligations
- data obligations
- API obligations
- security obligations
- design obligations

- ops / release obligations
- verification obligations

New artifact

- `kit_extraction.json`

BAQ-07 — Derived Build Inputs Pack

Create a normalized planning pack before generation.

Must include

- domain model summary
- subsystem map
- feature map
- API surface map
- storage/data map
- auth/RBAC map
- verification obligations
- ops obligations
- dependency assumptions
- risk points

New artifact

- `derived_build_inputs.json`

BAQ-08 — Repo Inventory Planner

Generate the expected repo/file inventory before writing code.

Must include

- expected directories
- expected modules
- required files
- optional files
- proof files
- test files

- manifest targets

New artifact

- `repo_inventory.json`

BAQ-09 — Requirement Trace Map

Add a trace layer that maps requirement sources to generation targets.

Must map

- requirement → work unit
- work unit → acceptance item
- acceptance item → target file(s)
- target file(s) → proof artifact(s)

New artifact

- `requirement_trace_map.json`

BAQ-10 — Build Sufficiency Policy

Replace broad expansion behavior with explicit sufficiency rules.

Policy rules

- generate only what the kit justifies
- suppress optional artifact families unless triggered
- enforce profile-aware generation
- reject placeholder completion for required implementation units
- mark inferred output explicitly

BAQ-11 — Failure Classification Engine

All failed or degraded builds should classify the reason.

Failure classes

- extraction failure
- planning failure
- inventory failure
- generation failure
- traceability failure
- verification failure
- packaging failure

Each failure record should include

- failing unit
- blocker type
- expected artifacts
- produced artifacts
- missing artifacts
- upstream dependency
- retry eligibility
- severity

BAQ-12 — Packaging Integrity Gate

Before kit packaging, reconcile all critical artifacts.

Must reconcile

- extraction report
- derived build inputs
- repo inventory
- generated artifacts
- requirement trace map
- proof artifacts
- final manifest

Packaging should fail if

- required outputs are missing
- trace map is incomplete
- hard-gate acceptance items lack proof
- manifest does not match produced outputs

---

7. Gate Upgrades

Existing weakness to fix

Current build quality issues suggest the agent can move forward without proving it consumed the full kit or met full output obligations.

New gates

G-BQ-01 — Extraction Completeness

Pass only if all required kit layers are extracted or explicitly ruled not applicable.

G-BQ-02 — Planning Completeness

Pass only if the derived build plan and repo inventory are present and valid.

G-BQ-03 — Requirement Coverage

Pass only if all hard requirements map to implementation or proof.

G-BQ-04 — Output Sufficiency

Pass only if required files exist and placeholders do not stand in for required implementation.

G-BQ-05 — Verification Integrity

Pass only if the required proof/test obligations are satisfied.

G-BQ-06 — Packaging Integrity

Pass only if the final manifest reconciles with actual artifacts and trace outputs.

---

8. Verification Expansion

BAQ-13 — Required Tests

Add or upgrade tests for:

- valid kit extraction
- partial kit rejection
- derived plan correctness
- repo inventory generation
- requirement trace generation
- missing-file detection
- placeholder rejection for required units
- acceptance-to-proof reconciliation
- packaging rejection on incomplete builds

BAQ-14 — Quality Metrics

Track quality through measurable signals.

Recommended metrics

- extraction coverage %
- requirement coverage %
- acceptance coverage %
- expected vs produced file variance
- placeholder ratio
- inferred artifact ratio
- proof completion %
- packaging pass rate

---

9. Data / Artifact Additions

New required runtime artifacts

- `kit_extraction.json`
- `derived_build_inputs.json`
- `repo_inventory.json`
- `requirement_trace_map.json`
- `build_quality_report.json`
- `generation_failure_report.json` when applicable

Updated existing artifacts

- run manifest
- artifact index
- stage reports
- verification outputs
- packaging outputs

---

10. Rollout Plan

Your registry already supports the right maintenance framing: use **MM-20 — Failure Triage Mode** first to isolate recurring defects, then roll the implementation as **MM-05 — Planned Upgrade**.

Response: Axion Updates Planning

Phase 1 — Failure Triage

- collect recent bad kits
- identify missing artifact families
- classify failure patterns
- baseline current coverage and pass rates

Phase 2 — Extraction + Planning

- ship kit extraction contract
- ship derived build inputs
- ship repo inventory planner

Phase 3 — Traceability + Sufficiency

- add requirement trace map
- add sufficiency policy
- block placeholder completion for required units

Phase 4 — Gate Hardening

- add new build quality gates
- upgrade packaging integrity checks
- enforce failure classification

Phase 5 — Verification Expansion

- replace weak or todo validation with full tests
- add quality metrics and regression tracking

---

11. Success Criteria

The update is successful when:

- build kits consistently include all required artifact families
- missing implementation obligations are caught before packaging
- every hard requirement has a traceable implementation or proof path
- placeholder-heavy builds are blocked
- packaging only passes verified kits
- triage reports clearly show why failed builds failed

---

12. Official Recommendation

The highest-value rule in this entire update is:

The Build Agent must not generate code until it has produced a valid extraction report, a derived build plan, and an expected repo inventory.

That is the change most likely to raise build-kit quality fast.

Next, I can turn this into the full Axion-style spec format:

BUILD_AGENT_QUALITY_UPGRADE_SPEC
with:

- inputs
- flow
- gates
- outputs
- proofs
- failure modes

BUILD_AGENT_QUALITY_UPGRADE_SPEC

BUILD_AGENT_QUALITY_UPGRADE_SPEC

0. Spec Identity

Spec ID: BAQ-SPEC-01
Title: Build Agent Quality Upgrade
Mode Class: MM-05 — Planned Upgrade
Support Mode: MM-20 — Failure Triage Mode
Primary Domain: Build Agent / Build Kit Generation / Verification / Packaging
Purpose: raise build-kit quality by enforcing full kit extraction, pre-generation planning, requirement traceability, stronger validation, and verification-backed packaging.

---

1. Problem Statement

The Build Agent is producing build kits with bugs, weak coverage, or incomplete implementation because generation can proceed before the system has proven:

- that it consumed the full authoritative kit
- that it derived a complete repo plan
- that requirements map to implementation targets
- that expected output inventory was satisfied
- that packaged kits are actually credible

This creates a failure pattern where builds can appear complete while still missing required modules, proof artifacts, or implementation depth.

---

2. Objective

The Build Agent must move from **best-effort generation** to **authority-driven generation**.

The upgraded Build Agent must:

1. extract all required build authority from the kit
2. derive normalized build inputs
3. create an expected repo inventory before code generation

4. trace requirements to implementation and proof targets
5. generate only sufficient justified output
6. block incomplete or low-confidence packaging
7. emit diagnostics that explain why a build degraded or failed

---

3. Success Conditions

This upgrade is complete when all of the following are true:

- required kit layers are extracted and recorded
- generation cannot begin without a valid derived build plan
- all hard requirements trace to implementation or proof
- expected-vs-produced inventory variance is enforced
- placeholder-only fulfillment is blocked for required units
- incomplete builds fail before packaging
- quality reports clearly explain missing coverage and blockers

---

4. Scope

In Scope

- build kit extraction
- normalized build input derivation
- repo inventory planning
- requirement traceability
- sufficiency rules
- validation gates
- failure taxonomy
- packaging integrity checks
- verification upgrades
- quality metrics

Out of Scope

- full UI redesign
- unrelated control plane redesign
- replacing the entire template system
- unrelated pipeline stage rewrites unless directly required by this spec

---

5. Inputs

5.1 Required Inputs

The Build Agent must accept a build authority bundle that may include:

- build brief
- canonical spec
- assumptions / unknowns
- standards resolution
- work breakdown
- acceptance map
- architecture pack
- implementation pack
- data pack
- API pack
- security pack
- design pack
- ops / runtime pack
- verification pack
- packaging constraints

5.2 Optional Inputs

- profile selection
- target stack selection
- deployment target rules
- repo conventions
- naming conventions
- environment constraints
- pre-existing workspace state

5.3 Input Validity Rule

No input may be treated as implicitly consumed.
Every required input must be marked as one of:

- `consumed`
- `deferred`
- `not_applicable`
- `invalid`

If marked `deferred`, `not_applicable`, or `invalid`, a reason is required.

---

6. Outputs

6.1 New Required Runtime Artifacts

The upgraded Build Agent must emit:

- `kit_extraction.json`
- `derived_build_inputs.json`
- `repo_inventory.json`
- `requirement_trace_map.json`
- `build_quality_report.json`

6.2 Conditional Runtime Artifacts

Emit when needed:

- `generation_failure_report.json`
- `coverage_gap_report.json`
- `placeholder_violation_report.json`
- `packaging_block_report.json`

6.3 Updated Existing Outputs

The following existing outputs must be updated to reference the new quality layer:

- stage report
- run manifest
- artifact index
- proof ledger
- packaging manifest
- verification summary

---

7. Core Operating Model

7.1 Authority Rule

The Build Agent must treat the build kit as the authoritative source of build intent.

Generation may not be driven by a summary-only interpretation when authoritative sections exist.

7.2 Plan-Before-Generate Rule

The Build Agent must not generate code or package artifacts until it has produced:

- a valid extraction report
- a valid derived build inputs artifact
- a valid repo inventory

7.3 Traceability Rule

Every required requirement path must resolve to one of:

- implementation target
- proof target
- explicit non-applicable ruling

7.4 Sufficiency Rule

The Build Agent must generate the **minimum sufficient artifact set** needed to satisfy:

- functional obligations
- structural obligations
- interface obligations
- verification obligations
- runtime / ops obligations
- packaging obligations

7.5 Packaging Trust Rule

Packaging is a trust boundary.
A build cannot package as complete unless required extraction, planning, generation, and proof conditions pass.

---

8. Execution Flow

BAQ-STEP-01 — Intake Load

Load build authority bundle and detect all available sections.

Produces

- input registry
- section presence map

Fail if

- no authoritative build source exists
- required foundational sections are unreadable

BAQ-STEP-02 — Kit Extraction

Extract required obligations from the full kit.

Must extract

- product/build mission
- domain model
- feature obligations
- data/storage obligations
- API obligations
- access/control obligations
- UI/interaction obligations
- runtime/ops obligations
- verification obligations
- packaging obligations

Produces

- `kit_extraction.json`

Fail if

- required sections are present but not extractable
- extraction leaves critical authority unmapped
- required sections are silently skipped

BAQ-STEP-03 — Derived Build Inputs

Normalize extracted authority into generation-ready inputs.

Must derive

- subsystem map
- feature-to-surface map
- domain entity map
- storage map
- API surface map
- auth/RBAC map
- workflow map
- verification map
- ops map
- dependency assumptions
- risk points

Produces

- `derived_build_inputs.json`

Fail if

- derived inputs are incomplete for required build areas
- critical implementation areas cannot be normalized

BAQ-STEP-04 — Repo Inventory Planning

Plan the expected repo and artifact inventory before file generation.

Must plan

- required directories
- required modules
- required files
- optional files
- proof files
- test files
- runtime/config files
- manifest entries

Produces

- `repo_inventory.json`

Fail if

- no credible file inventory can be derived
- required build areas have no file targets
- inventory conflicts cannot be resolved

BAQ-STEP-05 — Requirement Trace Mapping

Map authority to implementation and proof.

Must map

- requirement → work unit
- work unit → acceptance item
- acceptance item → file target
- file target → proof target

Produces

- `requirement_trace_map.json`

Fail if

- hard requirements are unmapped
- acceptance items have no file or proof targets
- trace graph is broken or partial

BAQ-STEP-06 — Sufficiency Evaluation

Apply artifact sufficiency rules before generation.

Must determine

- required artifact families
- optional artifact families
- suppressed artifact families
- inferred artifact families
- blocked artifact families

Produces

- sufficiency decision set inside `build_quality_report.json`

Fail if

- required build areas would be satisfied only by placeholder output
- optional artifacts are being used to mask required omissions

---

BAQ-STEP-07 — Generation

Generate files only from planned inventory and trace-backed obligations.

Generation constraints

- every generated required file must have trace support
- inferred files must be explicitly marked
- placeholder-only files cannot satisfy required implementation units
- generated structure must match planned inventory classes

Produces

- generated repo contents
- generation metadata
- missing artifact delta

Fail if

- required files are absent
- generation substitutes summaries for required implementation
- output diverges materially from planned inventory

BAQ-STEP-08 — Verification Reconciliation

Reconcile produced outputs against requirements, acceptance items, and proof obligations.

Must verify

- required files exist
- required proof targets exist
- acceptance coverage threshold is met
- inventory variance is within threshold
- placeholder violations are absent for required units

Produces

- verification status
- proof status
- coverage status

Fail if

- required coverage is below threshold
- proof obligations remain unresolved
- critical generated artifacts are missing

BAQ-STEP-09 — Packaging Integrity Evaluation

Reconcile all build-quality artifacts before packaging.

Must reconcile

- extraction report
- derived inputs
- repo inventory
- generated outputs
- trace map
- proof status
- final manifest

Produces

- final quality decision
- packaging authorization decision

Fail if

- manifest does not match actual outputs
- quality gates are not all passed
- hard requirements remain unimplemented or unproven

BAQ-STEP-10 — Final Reporting

Emit final quality and failure reports.

Must report

- extraction coverage
- requirement coverage
- acceptance coverage
- expected vs produced variance
- inferred artifact ratio
- placeholder ratio
- proof completion
- packaging eligibility
- blocker summary

---

9. Gate Model

G-BQ-01 — Extraction Completeness

Pass condition: all required kit layers are extracted or explicitly ruled not applicable with reason.

Evidence

- `kit_extraction.json`

G-BQ-02 — Derived Planning Completeness

Pass condition: normalized build inputs exist for all required build areas.

Evidence

- `derived_build_inputs.json`

G-BQ-03 — Inventory Planning Integrity

Pass condition: repo inventory covers all required feature, infrastructure, proof, and runtime obligations.

Evidence

- `repo_inventory.json`

G-BQ-04 — Requirement Coverage

Pass condition: all hard requirements map to implementation or proof targets.

Evidence

- `requirement_trace_map.json`

G-BQ-05 — Output Sufficiency

Pass condition: required outputs exist and no required unit is fulfilled only by placeholder structure.

Evidence

- generated files
- `build_quality_report.json`

G-BQ-06 — Verification Integrity

Pass condition: required tests/proofs/acceptance obligations meet threshold.

Evidence

- verification outputs
- proof refs
- quality report

G-BQ-07 — Packaging Integrity

Pass condition: final manifest reconciles with actual outputs and all prior gates passed.

Evidence

- final manifest
- packaging report
- quality report

---

10. Artifact Contracts

10.1 `kit_extraction.json`

Required fields

- `run_id`
- `spec_version`
- `source_sections`
- `required_sections`
- `extracted_sections`
- `section_statuses`
- `missing_sections`
- `deferred_sections`
- `non_applicable_sections`
- `invalid_sections`
- `critical_obligations`
- `extraction_warnings`

10.2 `derived_build_inputs.json`

Required fields

- `run_id`
- `spec_version`
- `subsystem_map`
- `feature_map`
- `domain_model`
- `storage_model`
- `api_surface`
- `auth_model`
- `ui_surface_map`
- `verification_obligations`
- `ops_obligations`
- `assumptions`
- `risks`

10.3 `repo_inventory.json`

Required fields

- `run_id`
- `spec_version`
- `inventory_profile`
- `required_directories`
- `required_files`
- `optional_files`
- `proof_files`
- `test_files`
- `config_files`
- `runtime_files`
- `manifest_targets`
- `expected_counts`

10.4 `requirement_trace_map.json`

Required fields

- `run_id`
- `spec_version`
- `requirements`
- `work_units`
- `acceptance_items`
- `file_targets`
- `proof_targets`
- `unmapped_requirements`
- `trace_gaps`
- `trace_summary`

10.5 `build_quality_report.json`

Required fields

- `run_id`
- `spec_version`
- `gate_results`
- `extraction_coverage_pct`
- `requirement_coverage_pct`
- `acceptance_coverage_pct`
- `inventory_variance`
- `placeholder_ratio`
- `inferred_artifact_ratio`
- `proof_completion_pct`
- `quality_warnings`
- `blocking_issues`
- `packaging_eligible`

10.6 `generation_failure_report.json`

Required fields

- `run_id`
- `failure_class`
- `failing_units`
- `missing_artifacts`
- `expected_artifacts`
- `produced_artifacts`
- `upstream_blockers`
- `severity`
- `retry_eligibility`
- `repair_hints`

---

11. Failure Taxonomy

BAQ-F-01 — Extraction Failure

A required section exists but cannot be extracted into obligations.

BAQ-F-02 — Planning Failure

Derived build inputs cannot produce a coherent subsystem or surface map.

BAQ-F-03 — Inventory Failure

Required output structure cannot be represented as a credible inventory.

BAQ-F-04 — Traceability Failure

Hard requirements, work units, or acceptance items are unmapped.

BAQ-F-05 — Generation Failure

Required planned files were not produced or were generated in unusable form.

BAQ-F-06 — Placeholder Fulfillment Failure

A required implementation unit was satisfied only by stub, scaffold, or placeholder content.

BAQ-F-07 — Verification Failure

Proof, acceptance, or required quality checks did not pass.

BAQ-F-08 — Packaging Failure

Manifest or packaging state conflicts with actual outputs or gate state.

---

12. Enforcement Rules

12.1 No Silent Omission

Any required kit layer omitted from extraction is a failure unless explicitly ruled not applicable.

12.2 No Summary-Only Compliance

A summary cannot substitute for a structured authority section when that section is present.

12.3 No Placeholder Completion

Placeholder output cannot satisfy a required implementation, proof, or runtime obligation.

12.4 No Unplanned Required File

A required output file must exist in the repo inventory before generation.

12.5 No Untraced Hard Requirement

A hard requirement without an implementation or proof path blocks packaging.

12.6 No Green Packaging on Red Gates

Packaging cannot succeed if any hard build-quality gate fails.

---

13. Quality Metrics

The Build Agent should track these metrics on every run:

- extraction coverage %
- requirement coverage %
- acceptance coverage %
- expected/produced inventory delta
- placeholder ratio
- inferred artifact ratio
- proof completion %
- packaging pass/fail
- failure-class frequency
- recurring missing-artifact families

These metrics are used to support both planned upgrade work and failure triage work under the maintenance registry modes already defined in your planning file.

---

14. Verification Plan

Required Test Classes

- complete-kit extraction test
- partial-kit rejection test

- invalid-section handling test
- derived-input normalization test
- repo inventory correctness test
- requirement trace completeness test
- placeholder rejection test
- expected-vs-produced variance test
- packaging block on failed gates test
- failure taxonomy classification test

Required Regression Focus

- missing infrastructure modules
- missing shared contracts
- missing proof artifacts
- missing verification files
- incomplete feature family generation
- manifest/output mismatch
- false-positive “complete” packaging

---

15. Rollout Plan

Phase 1 — MM-20 Failure Triage Mode

Use failure triage first to baseline the current defect pattern and collect representative bad build kits.

Phase outputs

- bad build sample set
- failure class baseline
- recurring omission matrix
- current metric baseline

Phase 2 — Extraction + Planning Foundation

Ship:

- `kit_extraction.json`
- `derived_build_inputs.json`
- `repo_inventory.json`

Phase 3 — Traceability Layer

Ship:

- `requirement_trace_map.json`
- unmapped requirement blocking
- proof target linking

Phase 4 — Gate Hardening

Ship:

- G-BQ-01 through G-BQ-07
- packaging trust boundary
- placeholder rejection

Phase 5 — Verification Expansion

Ship:

- integration tests
- regression suites
- quality metrics dashboard/reporting hooks

Phase 6 — MM-05 Planned Upgrade Rollout

Promote the new Build Agent quality model as the standard planned upgrade path.

---

16. Completion Criteria

This spec is complete when:

- all new required artifacts are emitted
- all hard gates are enforced
- incomplete builds fail before packaging
- requirement trace gaps are visible
- bad builds classify cleanly into failure taxonomy
- regression tests protect against recurring omission patterns
- packaging can be trusted as “quality-cleared”

---

17. Final Operating Rule

The Build Agent must not generate or package a build kit unless extraction, derived planning, repo inventory, and requirement traceability are all present and valid.

That is the core rule that raises build-kit quality.

DATA SCHEMA PACK

BUILD_AGENT_QUALITY_UPGRADE — DATA SCHEMA PACK

0. Schema Pack Identity

Pack ID: BAQ-SCHEMA-01
Parent Spec: BAQ-SPEC-01
Purpose: define the minimum data contracts required to enforce Build Agent quality, traceability, verification, and packaging integrity.

---

1. Schema Set

This update introduces six core schema surfaces:

1. `kit_extraction.json`
2. `derived_build_inputs.json`
3. `repo_inventory.json`
4. `requirement_trace_map.json`
5. `build_quality_report.json`
6. `generation_failure_report.json`

These are runtime enforcement artifacts, not optional notes.

---

2. Global Contract Rules

2.1 Shared Required Fields

Every schema in this pack must include:

- `schema_id`
- `schema_version`
- `run_id`
- `created_at`
- `producer`
- `status`

2.2 Shared Enum Rules

`status`

Allowed values:

- `ok`
- `warning`
- `failed`
- `blocked`

`severity`

Allowed values:

- `low`
- `medium`

- `high`
- `critical`

`applicability_status`

Allowed values:

- `consumed`
- `deferred`
- `not_applicable`
- `invalid`
- `missing`

---

3. KIT_EXTRACTION_SCHEMA

3.1 Purpose

Records exactly what the Build Agent extracted from the authoritative build kit.

3.2 File

`kit_extraction.json`

3.3 Contract


```
{
 "schema_id": "KIT_EXTRACTION_SCHEMA",
 "schema_version": "1.0.0",
 "run_id": "string",
 "created_at": "ISO-8601",
```

"producer": "build-agent",
"status": "ok|warning|failed|blocked",
"source_bundle_id": "string",
"source_sections": [
  {
    "section_id": "string",
    "section_name": "string",
    "required": true,
    "present": true
  }
],
"required_sections": ["string"],
"section_statuses": [
  {
    "section_id": "string",
    "status": "consumed|deferred|not_applicable|invalid|missing",
    "reason": "string|null",
    "obligation_count": 0,
    "warning_count": 0
  }
],
"critical_obligations": [
  {
    "obligation_id": "string",
    "category": "feature|data|api|security|ui|ops|verification|packaging",
    "summary": "string",
    "source_section_id": "string",
    "required": true
  }
],
"missing_sections": ["string"],
"deferred_sections": ["string"],
"non_applicable_sections": ["string"],
"invalid_sections": ["string"],
"extraction_warnings": [
  {
    "code": "string",
    "message": "string",
    "severity": "low|medium|high|critical"
  }
],
"metrics": {
  "required_section_count": 0,
  "consumed_required_section_count": 0,

```
  "critical_obligation_count": 0,
  "extraction_coverage_pct": 0
 }
}
```

3.4 Rules

- every required source section must appear in `section_statuses`
- `missing`, `invalid`, and `deferred` require a `reason`
- `critical_obligations` cannot be empty for a real build run
- `extraction_coverage_pct` is based on required sections, not optional sections

---

4. DERIVED_BUILD_INPUTS_SCHEMA

4.1 Purpose

Normalizes extracted authority into generation-ready planning data.

4.2 File

`derived_build_inputs.json`

4.3 Contract

```
{
 "schema_id": "DERIVED_BUILD_INPUTS_SCHEMA",
 "schema_version": "1.0.0",
 "run_id": "string",
 "created_at": "ISO-8601",
 "producer": "build-agent",
 "status": "ok|warning|failed|blocked",
 "build_profile": "lean|standard|enterprise|custom",
 "subsystem_map": [
  {
```

```
    "subsystem_id": "string",
    "name": "string",
    "purpose": "string",
    "required": true
  }
],
"feature_map": [
  {
    "feature_id": "string",
    "name": "string",
    "subsystem_id": "string",
    "priority": "critical|high|medium|low",
    "required": true
  }
],
"domain_model": {
  "entities": [
    {
      "entity_id": "string",
      "name": "string",
      "kind": "core|supporting|system",
      "required_fields": ["string"]
    }
  ],
  "relationships": [
    {
      "from": "string",
      "to": "string",
      "type": "one_to_one|one_to_many|many_to_many|dependency"
    }
  ]
},
"storage_model": {
  "stores": [
    {
      "store_id": "string",
      "type": "db|cache|blob|queue|file",
      "purpose": "string"
    }
  ],
  "data_flows": [
    {
      "from": "string",
      "to": "string",
```

```
      "mode": "read|write|sync|async"
    }
  ]
},
"api_surface": [
  {
    "endpoint_id": "string",
    "route": "string",
    "method": "GET|POST|PUT|PATCH|DELETE",
    "auth_required": true,
    "purpose": "string"
  }
],
"auth_model": {
  "enabled": true,
  "roles": ["string"],
  "permissions": [
    {
      "role": "string",
      "action": "string",
      "resource": "string"
    }
  ]
},
"ui_surface_map": [
  {
    "surface_id": "string",
    "name": "string",
    "type": "page|panel|tab|dialog|widget|api_only",
    "required": true
  }
],
"verification_obligations": [
  {
    "verification_id": "string",
    "target_type": "feature|api|data|security|ops|packaging",
    "target_id": "string",
    "required": true
  }
],
"ops_obligations": [
  {
    "ops_id": "string",
    "category": "logging|metrics|alerts|slo|config|deploy|backup",
```

```
    "required": true
  }
 ],
 "assumptions": ["string"],
 "risks": [
  {
    "risk_id": "string",
    "summary": "string",
    "severity": "low|medium|high|critical"
  }
 ]
}
```

4.4 Rules

- every required feature must map to a subsystem
- API routes that affect protected resources must align with `auth_model`
- `verification_obligations` must exist for all critical features
- `ui_surface_map` may include `api_only` when no direct UI is required

---

5. REPO_INVENTORY_SCHEMA

5.1 Purpose

Defines the expected file and module inventory before generation starts.

5.2 File

`repo_inventory.json`

5.3 Contract

```
{
 "schema_id": "REPO_INVENTORY_SCHEMA",
```

```
"schema_version": "1.0.0",
"run_id": "string",
"created_at": "ISO-8601",
"producer": "build-agent",
"status": "ok|warning|failed|blocked",
"inventory_profile": "lean|standard|enterprise|custom",
"required_directories": [
  {
    "path": "string",
    "purpose": "string"
  }
],
"required_files": [
  {
    "file_id": "string",
    "path": "string",
    "category": "ui|api|data|domain|security|ops|test|config|manifest|proof",
    "required": true,
    "trace_required": true
  }
],
"optional_files": [
  {
    "file_id": "string",
    "path": "string",
    "category": "ui|api|data|domain|security|ops|test|config|manifest|proof",
    "trigger_reason": "string"
  }
],
"proof_files": [
  {
    "file_id": "string",
    "path": "string",
    "proof_type": "test|report|manifest|ledger|evidence"
  }
],
"test_files": [
  {
    "file_id": "string",
    "path": "string",
    "test_type": "unit|integration|e2e|contract|verification"
  }
],
"config_files": [
```

```
    {
      "file_id": "string",
      "path": "string",
      "environment_scope": "shared|dev|test|prod"
    }
  ],
  "runtime_files": [
    {
      "file_id": "string",
      "path": "string",
      "runtime_role": "bootstrap|router|worker|service|pipeline|adapter"
    }
  ],
  "manifest_targets": ["string"],
  "expected_counts": {
    "required_file_count": 0,
    "optional_file_count": 0,
    "proof_file_count": 0,
    "test_file_count": 0
  }
}
```

5.4 Rules

- every `required_file` must have a stable `file_id`
- required files cannot exist only as category counts; they need concrete paths
- proof/test/config/runtime files may overlap with required files, but counts must still reconcile
- `manifest_targets` must cover all package-critical outputs

---

6. REQUIREMENT_TRACE_MAP_SCHEMA

6.1 Purpose

Maps authority to implementation and proof surfaces.

6.2 File

`requirement_trace_map.json`

6.3 Contract

```
{
 "schema_id": "REQUIREMENT_TRACE_MAP_SCHEMA",
 "schema_version": "1.0.0",
 "run_id": "string",
 "created_at": "ISO-8601",
 "producer": "build-agent",
 "status": "ok|warning|failed|blocked",
 "requirements": [
   {
     "requirement_id": "string",
     "summary": "string",
     "class": "functional|data|api|security|ui|ops|verification|packaging",
     "hard_requirement": true
   }
 ],
 "work_units": [
   {
     "work_unit_id": "string",
     "requirement_ids": ["string"],
     "summary": "string"
   }
 ],
 "acceptance_items": [
   {
     "acceptance_id": "string",
     "work_unit_id": "string",
     "summary": "string",
     "hard_gate": true
   }
 ],
 "file_targets": [
   {
     "target_id": "string",
     "acceptance_id": "string",
     "file_id": "string",
     "path": "string",
     "target_role": "implementation|interface|support|config|proof"
   }
```

```
  ],
  "proof_targets": [
    {
      "proof_target_id": "string",
      "acceptance_id": "string",
      "proof_type": "test|report|manifest|ledger|evidence",
      "path": "string"
    }
  ],
  "unmapped_requirements": ["string"],
  "trace_gaps": [
    {
      "gap_id": "string",
      "gap_type":
"missing_work_unit|missing_acceptance|missing_file_target|missing_proof_target",
      "source_id": "string",
      "severity": "low|medium|high|critical"
    }
  ],
  "trace_summary": {
    "requirement_count": 0,
    "mapped_requirement_count": 0,
    "hard_requirement_count": 0,
    "mapped_hard_requirement_count": 0,
    "acceptance_count": 0,
    "acceptance_with_file_targets": 0,
    "acceptance_with_proof_targets": 0
  }
}
```

6.4 Rules

- every hard requirement must map through the full chain or appear in `unmapped_requirements`
- every hard-gate acceptance item must have at least one file target
- proof targets are mandatory for all hard-gate acceptance items unless explicitly waived by policy
- `trace_summary` must be mechanically derivable from the arrays

7. BUILD_QUALITY_REPORT_SCHEMA

7.1 Purpose

Provides the final quality decision surface for the run.

7.2 File

`build_quality_report.json`

7.3 Contract

```
{
 "schema_id": "BUILD_QUALITY_REPORT_SCHEMA",
 "schema_version": "1.0.0",
 "run_id": "string",
 "created_at": "ISO-8601",
 "producer": "build-agent",
 "status": "ok|warning|failed|blocked",
 "gate_results": [
   {
     "gate_id": "G-BQ-01|G-BQ-02|G-BQ-03|G-BQ-04|G-BQ-05|G-BQ-06|G-BQ-07",
     "passed": true,
     "severity": "low|medium|high|critical",
     "message": "string"
   }
 ],
 "coverage": {
   "extraction_coverage_pct": 0,
   "requirement_coverage_pct": 0,
   "acceptance_coverage_pct": 0,
   "proof_completion_pct": 0
 },
 "inventory": {
   "expected_required_files": 0,
   "produced_required_files": 0,
   "missing_required_files": 0,
   "inventory_variance_pct": 0
 },
```

```
"quality_signals": {
  "placeholder_ratio": 0,
  "inferred_artifact_ratio": 0,
  "warning_count": 0,
  "blocking_issue_count": 0
},
"quality_warnings": [
  {
    "code": "string",
    "message": "string",
    "severity": "low|medium|high|critical"
  }
],
"blocking_issues": [
  {
    "issue_id": "string",
    "category": "extraction|planning|inventory|trace|generation|verification|packaging",
    "message": "string",
    "severity": "low|medium|high|critical"
  }
],
"packaging_eligible": true,
"final_decision": "approved|approved_with_warnings|blocked|failed"
}
```


7.4 Rules

- `packaging_eligible` must be false if any hard gate fails
- `final_decision` cannot be `approved` if `blocking_issue_count > 0`
- `inventory_variance_pct` must use required files as denominator
- `placeholder_ratio` must exclude non-required scaffolding unless policy says otherwise

---

8. GENERATION_FAILURE_REPORT_SCHEMA

8.1 Purpose

Captures failed or degraded generation outcomes in a repairable format.

8.2 File

`generation_failure_report.json`

8.3 Contract

```
{
 "schema_id": "GENERATION_FAILURE_REPORT_SCHEMA",
 "schema_version": "1.0.0",
 "run_id": "string",
 "created_at": "ISO-8601",
 "producer": "build-agent",
 "status": "failed|blocked|warning",
 "failure_class":
"extraction_failure|planning_failure|inventory_failure|traceability_failure|generation_failure|place
holder_fulfillment_failure|verification_failure|packaging_failure",
 "failing_units": [
   {
     "unit_id": "string",
     "unit_type": "requirement|work_unit|acceptance|file|proof|gate",
     "summary": "string",
     "severity": "low|medium|high|critical"
   }
 ],
 "expected_artifacts": [
   {
     "artifact_id": "string",
     "path": "string",
     "category": "ui|api|data|domain|security|ops|test|config|manifest|proof"
   }
 ],
```

```
 "produced_artifacts": [
   {
     "artifact_id": "string",
     "path": "string",
     "category": "ui|api|data|domain|security|ops|test|config|manifest|proof"
   }
 ],
 "missing_artifacts": [
   {
     "artifact_id": "string",
     "path": "string",
     "reason": "string"
   }
 ],
 "upstream_blockers": [
   {
     "blocker_id": "string",
     "source_stage": "extraction|planning|inventory|trace|generation|verification|packaging",
     "message": "string"
   }
 ],
 "retry_eligibility": "safe_retry|repair_then_retry|manual_review_required|do_not_retry",
 "repair_hints": ["string"]
}
```

8.4 Rules

- `failure_class` must come from a controlled enum
- `missing_artifacts` must be non-empty for inventory/generation failures
- `repair_hints` should be machine-usable and operator-usable
- `do_not_retry` is required when failure source is corrupt authority or invalid policy state

---

9. Cross-Schema Relations

9.1 Required Foreign-Key Style Links

These are logical references even if you do not implement strict foreign keys:

- `kit_extraction.critical_obligations[].obligation_id`
  → may feed `derived_build_inputs` items

- `repo_inventory.required_files[].file_id`
  → referenced by `requirement_trace_map.file_targets[].file_id`

- `requirement_trace_map.acceptance_items[].acceptance_id`
  → referenced by `file_targets` and `proof_targets`

- `build_quality_report.gate_results[]`
  → based on all prior schema outputs

9.2 Reconciliation Rules

- every file target in trace map must exist in repo inventory
- every missing required file in quality report must also exist in repo inventory
- any blocked packaging decision must reconcile with gate results
- failure report artifact IDs must reference planned inventory IDs where applicable

---

10. Minimum Validation Rules

10.1 Structural Validation

Each schema must pass:

- valid JSON
- required fields present
- enum validity

- type validity
- non-null required arrays where specified

10.2 Semantic Validation

Each schema must also pass:

- coverage math consistency
- count reconciliation
- trace-map referential integrity
- inventory-to-quality consistency
- gate-to-final-decision consistency

10.3 Failure Rules

A schema is invalid if:

- required IDs are missing
- percentages exceed range
- hard requirements are omitted silently
- required counts do not match arrays
- artifact references point to undefined targets

---

11. Recommended Type Pack

11.1 Shared Types

type RunStatus = "ok" | "warning" | "failed" | "blocked";

```
type Severity = "low" | "medium" | "high" | "critical";

type ApplicabilityStatus =
 | "consumed"
 | "deferred"
 | "not_applicable"
 | "invalid"
 | "missing";
```

11.2 Example Gate Type

```
type BuildQualityGateId =
 | "G-BQ-01"
 | "G-BQ-02"
 | "G-BQ-03"
 | "G-BQ-04"
 | "G-BQ-05"
 | "G-BQ-06"
 | "G-BQ-07";
```

11.3 Example Failure Enum

```
type GenerationFailureClass =
 | "extraction_failure"
 | "planning_failure"
 | "inventory_failure"
 | "traceability_failure"
 | "generation_failure"
 | "placeholder_fulfillment_failure"
 | "verification_failure"
 | "packaging_failure";
```

---

12. Implementation Notes

This schema pack should be enforced in three layers:

1. **compile-time types**

2. **runtime JSON schema validation**

3. **gate-level semantic validation**

Do not rely on type safety alone.
Bad build kits are often structurally valid but semantically incomplete.

---

13. Completion Condition

This schema pack is complete when:

- all six schemas exist
- the Build Agent can emit them
- validation rejects incomplete or contradictory quality state
- gate logic reads from these artifacts directly
- packaging cannot bypass them

IMPLEMENTATION PASS PLAN

BUILD_AGENT_QUALITY_UPGRADE — IMPLEMENTATION PASS PLAN

0. Plan Identity

Plan ID: BAQ-IMP-01
Parent Spec: BAQ-SPEC-01
Parent Schema Pack: BAQ-SCHEMA-01
Purpose: define the execution order for implementing the Build Agent quality upgrade without drifting scope or breaking packaging trust.

---

1. Implementation Strategy

This update should be built in controlled passes.

The order matters.

You do **not** want to start with UI, cosmetic reporting, or broad prompt tuning.
You want to lock the enforcement spine first:

1. extraction
2. normalization
3. inventory planning
4. traceability
5. sufficiency enforcement
6. gate hardening
7. verification expansion
8. reporting and operator surfaces

---

2. Pass Map

Pass 1 — Extraction Foundation

Goal

Make the Build Agent prove what it consumed from the build kit.

Deliverables

- `KIT_EXTRACTION_SCHEMA`
- `kit_extraction.json` emitter
- section presence scanner
- required section registry
- section status resolver
- extraction warning surface

Required behavior

- detect all authoritative sections
- classify each section as consumed, deferred, not applicable, invalid, or missing
- record critical obligations extracted from each section
- fail on silent omission of required sections

Exit criteria

- a build run cannot proceed without a valid extraction artifact

- required section coverage is measurable
- missing and invalid sections are explicit

Pass 2 — Derived Build Inputs

Goal

Convert raw extracted authority into generation-ready planning inputs.

Deliverables

- `DERIVED_BUILD_INPUTS_SCHEMA`
- `derived_build_inputs.json` emitter
- subsystem map builder
- feature map builder
- API surface derivation
- storage/data derivation
- auth/RBAC derivation
- verification obligations derivation
- ops obligations derivation

Required behavior

- all critical extracted obligations must resolve into normalized planning surfaces
- every required feature must attach to a subsystem or declared build area
- critical risk points and assumptions must be carried forward

Exit criteria

- generation no longer depends on loosely interpreted source summaries
- normalized planning state exists for all required areas

Pass 3 — Repo Inventory Planner

Goal

Define the expected repo structure before code generation begins.

Deliverables

- `REPO_INVENTORY_SCHEMA`
- `repo_inventory.json` emitter
- required directory planner
- required file planner
- optional file planner
- proof/test/config/runtime inventory planners
- manifest target planner

Required behavior

- required files must be concrete paths, not category placeholders
- planned inventory must cover feature, infrastructure, proof, runtime, and packaging needs
- each required file must have a stable identifier

Exit criteria

- every build has a planned file inventory
- generation can be measured against expected structure
- packaging-critical outputs are known in advance

Pass 4 — Requirement Traceability

Goal

Map requirements all the way to implementation and proof surfaces.

Deliverables

- `REQUIREMENT_TRACE_MAP_SCHEMA`
- `requirement_trace_map.json` emitter
- requirement ingestor
- work-unit mapper
- acceptance linker
- file target linker
- proof target linker
- gap detector

Required behavior

- every hard requirement must trace forward
- every hard-gate acceptance item must have file targets
- proof obligations must be visible and linkable

- trace gaps must be explicit and severity-rated

Exit criteria

- no hard requirement disappears silently
- unmapped requirements block completion
- file generation has trace authority behind it

Pass 5 — Sufficiency Engine

Goal

Stop overgeneration and stop fake completion.

Deliverables

- sufficiency policy engine
- artifact family classifier
- required vs optional artifact decision logic
- inferred artifact flagging
- placeholder fulfillment detector

Required behavior

- only generate artifacts justified by the kit and planning state
- optional artifacts cannot hide missing required output
- placeholders cannot satisfy required implementation units
- inferred files must be marked and count toward quality signals

Exit criteria

- build output becomes more disciplined
- placeholder-heavy kits are detectable
- minimum sufficient output policy is enforced

Pass 6 — Generation Alignment

Goal

Make actual file generation obey inventory and trace rules.

Deliverables

- generator preflight validator
- file generation planner hooks
- required-file reconciliation
- generated-output delta tracker
- missing artifact detector

Required behavior

- generation starts only after extraction, derived inputs, inventory, and trace are valid
- required files must be trace-backed
- produced outputs must reconcile against planned outputs
- missing required files trigger degradation or failure

Exit criteria

- generation is no longer freeform
- output drift becomes measurable
- required file gaps are surfaced immediately

Pass 7 — Gate Hardening

Goal

Turn quality signals into enforced gates.

Deliverables

- G-BQ-01 extraction completeness
- G-BQ-02 derived planning completeness
- G-BQ-03 inventory planning integrity
- G-BQ-04 requirement coverage
- G-BQ-05 output sufficiency
- G-BQ-06 verification integrity
- G-BQ-07 packaging integrity

Required behavior

- hard gate failure blocks packaging
- gate evidence must come from runtime artifacts, not informal logs
- gate results must be written into the quality report

Exit criteria

- packaging cannot go green on incomplete builds
- quality state is machine-enforced
- run completion reflects actual build readiness

Pass 8 — Build Quality Report

Goal

Create one authoritative decision surface for build quality.

Deliverables

- `BUILD_QUALITY_REPORT_SCHEMA`
- `build_quality_report.json` emitter
- coverage calculators
- inventory variance calculator
- placeholder ratio calculator
- inferred artifact ratio calculator
- final decision resolver

Required behavior

- quality report must summarize all gate states
- coverage math must reconcile with source artifacts
- packaging eligibility must reflect hard gate state
- final decision must be deterministic

Exit criteria

- every run has one final quality artifact
- build trust status is easy to inspect
- downstream systems can rely on one decision contract

Pass 9 — Failure Reporting

Goal

Classify degraded and failed runs into repairable categories.

Deliverables

- `GENERATION_FAILURE_REPORT_SCHEMA`
- `generation_failure_report.json` emitter
- failure taxonomy resolver
- failing unit collector
- upstream blocker linker
- retry eligibility logic
- repair hints generator

Required behavior

- failures must classify into controlled categories
- missing artifacts must be recorded concretely
- retry decisions must distinguish safe retry from repair-first cases

- failure reporting must link back to inventory and trace state where possible

Exit criteria

- bad kits are diagnosable
- recurring failure patterns can be trended
- failure triage becomes structured instead of manual guesswork

---

Pass 10 — Verification Expansion

Goal

Replace weak or placeholder validation with real enforcement coverage.

Deliverables

- extraction coverage tests
- derived planning tests
- repo inventory tests
- trace completeness tests
- placeholder rejection tests
- expected-vs-produced variance tests
- gate blocking tests
- packaging rejection tests
- failure taxonomy tests

Required behavior

- integration coverage must reflect real build-kit failure modes
- tests must include partial and malformed build kits
- tests must include missing required file scenarios
- tests must include false-green packaging prevention

Exit criteria

- recurring bugs are protected by regression tests
- core enforcement spine has coverage
- packaging trust boundary is test-backed

Pass 11 — Manifest and Packaging Reconciliation

Goal

Make final packaging reflect actual quality-cleared output.

Deliverables

- packaging preflight reconciliation
- manifest reconciliation logic
- package block reporting
- artifact index alignment updates

Required behavior

- final manifest must match actual generated outputs
- package-critical artifacts must exist before packaging

- packaging must read gate state directly
- missing proof or trace artifacts must block complete packaging

Exit criteria

- package completion becomes credible
- zipped output aligns with declared manifest state
- incomplete kits cannot masquerade as complete deliverables

Pass 12 — Operator Diagnostics Surface

Goal

Make it easier to inspect why a build passed, degraded, or failed.

Deliverables

- operator-readable quality summary
- operator-readable failure summary
- omission matrix
- gate breakdown summary
- coverage trend hooks

Required behavior

- diagnostics must read from the same runtime artifacts as the gates
- summaries must separate warning state from blocking state
- recurring missing artifact families should be easy to identify

Exit criteria

- triage becomes faster
- upgrade effectiveness becomes visible
- operator trust improves

---

3. Recommended Build Order Inside the Codebase

First block to implement

- schema/type definitions
- extraction scanner
- extraction emitter

Second block

- derived build inputs compiler
- repo inventory planner

Third block

- trace map engine
- sufficiency engine

Fourth block

- generator preflight enforcement
- output reconciliation

Fifth block

- hard gates
- quality report
- failure report

Sixth block

- packaging reconciliation
- verification expansion
- operator diagnostics

---

4. Dependency Rules

Hard dependencies

- Pass 2 depends on Pass 1
- Pass 3 depends on Pass 2
- Pass 4 depends on Pass 3
- Pass 5 depends on Pass 4
- Pass 6 depends on Passes 1–5
- Pass 7 depends on Passes 1–6

- Pass 8 depends on Passes 1–7
- Pass 9 depends on Passes 3–8
- Pass 10 depends on Passes 1–9
- Pass 11 depends on Passes 7–10
- Pass 12 depends on Passes 8–11

Do not reorder

Do not start with:

- UI polish
- packaging cosmetics
- prompt-only tuning
- broad template rewrites

Those are downstream, not foundational.

---

5. Implementation Rules

Rule 1 — Enforce before optimizing

Do not optimize generation until extraction, planning, and traceability are enforced.

Rule 2 — Runtime artifacts are the source of truth

All gates and packaging decisions must read from emitted artifacts, not transient memory or logs.

Rule 3 — Required files must be concrete

A required file obligation must resolve to an actual planned path.

Rule 4 — Quality must be deterministic

Final decision logic cannot depend on vague heuristics alone.

Rule 5 — Packaging is last

Packaging logic should be one of the last enforcement layers wired in, not the first.

---

6. Milestone Set

Milestone A — Extraction Locked

Complete when:

- `kit_extraction.json` exists
- section status coverage is enforced
- missing sections are visible

Milestone B — Planning Locked

Complete when:

- `derived_build_inputs.json` exists
- `repo_inventory.json` exists
- required output structure is known before generation

Milestone C — Trace Locked

Complete when:

- `requirement_trace_map.json` exists
- hard requirements cannot vanish silently

Milestone D — Quality Locked

Complete when:

- build quality report exists
- hard gates block packaging
- placeholder completion is rejected

Milestone E — Packaging Trust Locked

Complete when:

- manifest reconciles with actual outputs
- incomplete kits cannot package as complete

Milestone F — Regression Locked

Complete when:

- recurring defect classes have tests
- false-green build paths are covered

---

7. Suggested Work Sequence for You

If you want the cleanest execution path, the next build passes should be done in this order:

1. Extraction pass
2. Derived inputs pass

3. **Repo inventory pass**
4. **Requirement trace pass**
5. **Sufficiency pass**
6. **Gate integration pass**
7. **Quality/failure report pass**
8. **Verification and packaging pass**

That is the shortest path to meaningfully better build kits.

---

8. Completion Condition

This implementation plan is complete when:

- the Build Agent cannot generate without validated planning artifacts
- the Build Agent cannot package without passed quality gates
- failed runs produce structured failure artifacts
- build quality is measurable, traceable, and enforced

HOOK SPEC

BUILD_AGENT_QUALITY_UPGRADE — HOOK SPEC

0. Hook Spec Identity

Spec ID: BAQ-HOOK-01
Parent Spec: BAQ-SPEC-01
Parent Plan: BAQ-IMP-01
Purpose: define where the Build Agent quality upgrade attaches to the existing build flow, what each hook consumes and produces, and what conditions block forward motion.

---

1. Hook Model

This update should be added as a set of **quality enforcement hooks** around the Build Agent flow.

These hooks are not cosmetic observers.
They are control points.

Each hook must do one or more of the following:

- inspect upstream build state
- emit a runtime artifact
- enforce a quality rule
- attach evidence to the run
- block the next step when the quality state is invalid

---

2. Hook Set Overview

Hook list

1. `onBuildAuthorityLoaded`
2. `onKitExtractionStart`
3. `onKitExtractionComplete`
4. `onDerivedInputsBuild`
5. `onRepoInventoryPlan`
6. `onRequirementTraceBuild`
7. `onSufficiencyEvaluation`
8. `beforeGenerationStart`
9. `onFileGenerated`
10. `onGenerationComplete`
11. `onVerificationReconcile`
12. `beforePackaging`
13. `onPackagingDecision`
14. `onBuildQualityFinalize`
15. `onGenerationFailure`

---

3. Hook Contracts

3.1 `onBuildAuthorityLoaded`

Purpose

Fire when the Build Agent has loaded the authoritative build inputs and identified the source bundle.

Timing

Immediately after build authority load, before extraction begins.

Consumes

- build authority bundle
- source section registry
- run context
- build profile if present

Produces

- section presence map
- source bundle identity
- initial quality context

Required actions

- scan all available authority sections
- register which sections are required for this run
- record source bundle metadata
- prepare extraction workset

Block conditions

Block if:

- no authoritative bundle is present
- the source bundle is unreadable
- foundational required sections are absent

3.2 `onKitExtractionStart`

Purpose

Initialize extraction state and begin structured obligation capture.

Timing

After authority load, before extraction work begins.

Consumes

- source section registry
- extraction rules
- applicability rules

Produces

- open extraction session
- section work queue

Required actions

- mark extraction start
- enumerate sections to inspect

- classify extraction-critical sections
- initialize section status defaults

Block conditions

Do not block here unless the extraction session cannot initialize.

3.3 `onKitExtractionComplete`

Purpose

Resolve extraction results and emit `kit_extraction.json`.

Timing

Immediately after extraction finishes.

Consumes

- extracted obligations
- section statuses
- extraction warnings
- required section registry

Produces

- `kit_extraction.json`
- extraction coverage signals
- extraction gate evidence

Required actions

- finalize section statuses
- compute extraction coverage
- record missing/deferred/invalid sections
- emit critical obligations
- attach warnings and blockers

Block conditions

Block forward progress if:

- required sections are missing without valid non-applicable reason
- critical obligations could not be extracted
- silent omission occurred
- extraction artifact is invalid

3.4 `onDerivedInputsBuild`

Purpose

Compile normalized planning inputs from extracted authority.

Timing

After successful extraction, before repo inventory planning.

Consumes

- `kit_extraction.json`
- extracted obligation set

- build profile
- normalization rules

Produces

- `derived_build_inputs.json`

Required actions

- build subsystem map
- build feature map
- build entity/data model
- derive API/auth/UI/ops/verification surfaces
- carry forward assumptions and risk points

Block conditions

Block if:

- critical obligations cannot be normalized
- required features have no planning surface
- required build areas remain undefined
- derived input artifact is invalid

3.5 `onRepoInventoryPlan`

Purpose

Create the expected repo/file inventory before generation.

Timing

After derived inputs build, before trace mapping or generation.

Consumes

- `derived_build_inputs.json`
- inventory planning rules
- profile-specific file rules
- packaging target rules

Produces

- `repo_inventory.json`

Required actions

- define required directories
- define required files
- define optional files
- define proof/test/config/runtime files
- define manifest targets
- compute expected counts

Block conditions

Block if:

- required build areas have no file targets
- required files are represented only as vague categories

- package-critical outputs are undefined
- inventory artifact is invalid

3.6 `onRequirementTraceBuild`

Purpose

Create full forward traceability from requirements to implementation and proof.

Timing

After repo inventory planning, before sufficiency evaluation.

Consumes

- requirements/work units/acceptance inputs
- `repo_inventory.json`
- extracted obligations
- trace rules

Produces

- `requirement_trace_map.json`

Required actions

- map requirement to work unit
- map work unit to acceptance item
- map acceptance item to file targets
- map acceptance item to proof targets

- record unmapped requirements and trace gaps

Block conditions

Block if:

- any hard requirement is unmapped
- any hard-gate acceptance item lacks file targets
- proof-required acceptance items lack proof targets
- trace artifact is invalid

3.7 `onSufficiencyEvaluation`

Purpose

Decide which artifact families are required, optional, suppressed, inferred, or blocked.

Timing

After trace build, before generation preflight.

Consumes

- `kit_extraction.json`
- `derived_build_inputs.json`
- `repo_inventory.json`
- `requirement_trace_map.json`
- sufficiency policy rules

Produces

- sufficiency decision set
- early quality signals

Required actions

- classify artifact families
- detect optional-overreach
- detect placeholder-risk areas
- flag inferred output
- record blocked artifact families

Block conditions

Block if:

- a required area would be satisfied only by placeholder output
- optional artifact families are being used to mask missing required areas
- no minimum sufficient set can be established

3.8 `beforeGenerationStart`

Purpose

Run the final preflight check before any files are generated.

Timing

Immediately before generation begins.

Consumes

- all prior build-quality artifacts
- generation rules
- preflight enforcement rules

Produces

- generation authorization decision

Required actions

- verify extraction passed
- verify derived inputs passed
- verify inventory exists
- verify trace exists
- verify sufficiency passed
- verify no unresolved hard blockers remain

Block conditions

Block if any upstream hard condition failed.

This is the main “no freeform generation” hook.

3.9 `onFileGenerated`

Purpose

Track each generated file against planned inventory and trace expectations.

Timing

Per file, during generation.

Consumes

- generated file metadata
- `repo_inventory.json`
- `requirement_trace_map.json`

Produces

- per-file generation evidence
- generated/planned match state
- inferred artifact flag when applicable

Required actions

- record generated path
- mark whether file was planned
- mark whether file is required or optional
- attach trace linkage where available
- mark inferred/unplanned generation
- record placeholder warning if detected

Block conditions

Do not necessarily block on a single file event, but record conditions that may fail the run at completion.

3.10 `onGenerationComplete`

Purpose

Reconcile produced outputs against planned outputs after generation ends.

Timing

Immediately after generation completes.

Consumes

- generated file registry
- `repo_inventory.json`
- `requirement_trace_map.json`

Produces

- missing artifact delta
- generated/planned variance data
- updated quality signals

Required actions

- compute required files produced
- compute missing required files
- compute inventory variance
- identify unplanned generated files
- identify placeholder fulfillment violations

Block conditions

Block if:

- required files are missing

- inventory variance exceeds policy
- placeholder-only fulfillment exists for required units
- proof-required files are absent

3.11 `onVerificationReconcile`

Purpose

Reconcile implementation state with acceptance and proof obligations.

Timing

After generation complete, before packaging.

Consumes

- generated outputs
- proof/test outputs
- `requirement_trace_map.json`
- verification policy rules

Produces

- acceptance coverage state
- proof completion state
- verification gate evidence

Required actions

- verify acceptance coverage

- verify proof targets exist
- reconcile missing proofs
- identify unresolved hard-gate items

Block conditions

Block if:

- hard-gate acceptance items are unresolved
- proof completion is below required threshold
- required verification outputs are missing

3.12 `beforePackaging`

Purpose

Run the packaging trust check before any final package is assembled.

Timing

Immediately before packaging starts.

Consumes

- all quality artifacts
- manifest targets
- packaging rules

Produces

- packaging preflight decision

Required actions

- reconcile manifest targets
- verify all hard gates passed
- verify package-critical artifacts exist
- verify final package is eligible

Block conditions

Block if:

- any hard gate failed
- required trace/proof/inventory artifacts are missing
- final manifest cannot reconcile with actual outputs

3.13 `onPackagingDecision`

Purpose

Record whether packaging was approved, blocked, or failed.

Timing

At the packaging decision point.

Consumes

- packaging preflight state
- manifest reconciliation results
- gate results

Produces

- packaging decision record
- package block reasons if applicable

Required actions

- record final packaging decision
- record reasons for block/failure
- link package decision to gate evidence

Block conditions

This is the decision hook itself.
If blocked, no complete package may be emitted.

3.14 `onBuildQualityFinalize`

Purpose

Emit the final authoritative quality decision for the run.

Timing

After packaging decision, or after packaging block.

Consumes

- gate results
- coverage metrics
- inventory metrics
- proof metrics
- blocker list

Produces

- `build_quality_report.json`

Required actions

- resolve final gate states
- compute coverage metrics
- compute variance metrics
- compute placeholder and inferred artifact ratios
- determine packaging eligibility
- determine final decision

Block conditions

This hook should not block.
It should always emit a final quality report, even on failed runs.

3.15 `onGenerationFailure`

Purpose

Capture degraded or failed runs in a structured repairable form.

Timing

Any time the run transitions to failed, blocked, or severely degraded state.

Consumes

- failure state
- blocker state

- missing artifact state
- upstream hook context

Produces

- `generation_failure_report.json`

Required actions

- classify failure type
- record failing units
- record missing artifacts
- record upstream blockers
- determine retry eligibility
- emit repair hints

Block conditions

This hook should never be skipped on a hard failure.

---

4. Hook Ordering

Required order

1. `onBuildAuthorityLoaded`
2. `onKitExtractionStart`
3. `onKitExtractionComplete`

4. `onDerivedInputsBuild`
5. `onRepoInventoryPlan`
6. `onRequirementTraceBuild`
7. `onSufficiencyEvaluation`
8. `beforeGenerationStart`
9. `onFileGenerated`
10. `onGenerationComplete`
11. `onVerificationReconcile`
12. `beforePackaging`
13. `onPackagingDecision`
14. `onBuildQualityFinalize`

Failure side path

- `onGenerationFailure` may fire from any blocking point after authority load

---

5. Hook State Rules

5.1 Upstream Artifact Rule

A hook may not assume upstream state exists unless the upstream artifact or decision is present and valid.

5.2 Deterministic Evidence Rule

Each blocking hook must record why it blocked and what artifact or condition triggered the block.

5.3 Idempotency Rule

Hooks that emit artifacts should be safe to re-run for the same run context if no upstream state changed.

5.4 Non-Bypass Rule

Packaging and final quality hooks must not be reachable by bypassing failed upstream hard hooks.

---

6. Hook Interfaces

6.1 Common Hook Context

Each hook should receive a shared context object with at least:

- `run_id`
- `build_profile`
- `workspace_id`
- `source_bundle_id`
- `stage_id`
- `upstream_artifact_refs`
- `policy_refs`

- `timestamp`
- `mode_class`

6.2 Common Hook Result

Each hook should return:

- `hook_name`
- `status`
- `passed`
- `blocking`
- `artifact_refs`
- `warnings`
- `errors`
- `metrics`
- `next_action`

---

7. Example Hook Type Shapes

```
type BuildQualityHookStatus = "ok" | "warning" | "failed" | "blocked";

type BuildQualityHookResult = {
 hook_name: string;
 status: BuildQualityHookStatus;
 passed: boolean;
 blocking: boolean;
 artifact_refs: string[];
```

```
 warnings: Array<{ code: string; message: string }>;
 errors: Array<{ code: string; message: string }>;
 metrics?: Record<string, number>;
 next_action?:
   | "continue"
   | "retry"
   | "repair_then_retry"
   | "halt_generation"
   | "halt_packaging";
};

type BuildQualityHookContext = {
 run_id: string;
 build_profile: string;
 workspace_id?: string;
 source_bundle_id: string;
 stage_id?: string;
 upstream_artifact_refs: string[];
 policy_refs: string[];
 timestamp: string;
 mode_class: string;
};
```

---

8. Block Matrix

Hooks that may hard-block generation

- `onKitExtractionComplete`
- `onDerivedInputsBuild`
- `onRepoInventoryPlan`
- `onRequirementTraceBuild`
- `onSufficiencyEvaluation`
- `beforeGenerationStart`

Hooks that may hard-block packaging

- `onGenerationComplete`
- `onVerificationReconcile`
- `beforePackaging`
- `onPackagingDecision`

Hooks that must always emit even on failure

- `onBuildQualityFinalize`
- `onGenerationFailure`

---

9. Recommended Wiring Strategy

First wiring group

- `onBuildAuthorityLoaded`
- `onKitExtractionStart`
- `onKitExtractionComplete`

Second wiring group

- `onDerivedInputsBuild`
- `onRepoInventoryPlan`

- `onRequirementTraceBuild`

Third wiring group

- `onSufficiencyEvaluation`
- `beforeGenerationStart`
- `onGenerationComplete`

Fourth wiring group

- `onVerificationReconcile`
- `beforePackaging`
- `onPackagingDecision`
- `onBuildQualityFinalize`
- `onGenerationFailure`

This keeps the hook rollout aligned with the implementation pass order.

---

10. Completion Condition

This hook spec is complete when:

- every critical quality control point has a named hook
- every hook has defined inputs, outputs, and block conditions
- generation cannot start without passing preflight hooks

- packaging cannot complete without passing trust hooks
- failed runs always emit failure and final quality artifacts

PAGE SKELETON

BUILD_AGENT_QUALITY_UPGRADE — PAGE SKELETON

0. Page Skeleton Identity

Doc ID: BAQ-PAGE-01
Parent Spec: BAQ-SPEC-01
Parent Hook Spec: BAQ-HOOK-01
Purpose: define the operator-facing page structure for inspecting Build Agent quality, diagnosing weak build kits, and understanding why a build passed, degraded, or failed.

This is not a cosmetic dashboard.
This page is the operator control surface for build quality.

---

1. Page Purpose

The Build Agent quality page should let the operator answer these questions quickly:

- did the Build Agent consume the full build kit
- what did it plan to build
- what did it actually generate
- what requirements were not covered
- what gates failed
- why was packaging blocked
- what should be fixed first

---

2. Recommended Page Name

Build Quality

Alternate acceptable names

- Build Quality Control
- Build Integrity
- Build Kit Quality
- Build Quality Console

Best fit for clarity:
Build Quality

---

3. Route / Placement

Recommended route

`/build-quality`

Recommended navigation placement

Place under the Build Agent / Test / Assembly operating area, not buried under generic analytics.

This page should be treated as a primary operational page, not a passive report page.

---

4. Page Layout Overview

Primary layout

Use a multi-zone layout with:

Top bar

- run selector
- assembly/build selector
- status chip
- packaging eligibility chip
- final decision chip
- quick actions

Main summary band

- extraction coverage
- requirement coverage
- acceptance coverage
- proof completion
- inventory variance
- placeholder ratio

Main workspace

Tab or section-based detail panes:

1. Overview
2. Extraction
3. Derived Plan
4. Repo Inventory

5. Traceability
6. Generation Delta
7. Gates
8. Failures
9. Packaging
10. History / Trends

Right-side utility rail

- blocker summary
- top missing artifacts
- next recommended fix
- recent quality changes
- export/download actions

---

5. Top Bar Skeleton

5.1 Run Context Cluster

Show:

- build/assembly name
- run ID
- build profile
- source bundle ID

- timestamp
- mode class

5.2 Decision Cluster

Show:

- final decision
- packaging eligible yes/no
- overall status
- failure class if applicable

5.3 Quick Actions

Recommended actions:

- view quality report
- view failure report
- view trace gaps
- view missing files
- view packaging blockers
- export full quality bundle

---

6. Overview Section

Purpose

Give the operator the full run health at a glance.

Cards

Card 1 — Final Build Decision

Shows:

- approved / approved with warnings / blocked / failed
- one-line reason
- packaging status

Card 2 — Coverage Metrics

Shows:

- extraction coverage %
- requirement coverage %
- acceptance coverage %
- proof completion %

Card 3 — Output Integrity

Shows:

- expected required files
- produced required files
- missing required files
- inventory variance %

Card 4 — Quality Signals

Shows:

- placeholder ratio
- inferred artifact ratio
- warning count
- blocking issue count

Card 5 — Build Readiness Verdict

Shows:

- ready
- degraded
- blocked
- failed

Summary table

A compact summary row set:

- source bundle
- required sections
- extracted sections
- hard requirements
- mapped hard requirements
- package-critical artifacts
- manifest reconciliation result

---

7. Extraction Section

Purpose

Show whether the Build Agent consumed the full build authority.

Panels

Panel A — Section Coverage

Table:

- section name
- required
- present
- status
- obligation count
- warnings
- reason

Panel B — Critical Obligations

Grouped list by category:

- feature
- data
- api
- security
- ui

- ops
- verification
- packaging

Panel C — Extraction Warnings

List:

- warning code
- message
- severity
- source section

Panel D — Missing / Deferred / Invalid

Three separate grouped lists.

Primary operator question answered

“What did the Build Agent fail to consume?”

---

8. Derived Plan Section

Purpose

Show the normalized plan the generator should follow.

Panels

Panel A — Subsystem Map

Cards or table:

- subsystem
- purpose
- required
- linked features

Panel B — Feature Map

Table:

- feature
- subsystem
- priority
- required
- linked surfaces

Panel C — Surface Summary

Grouped counts:

- API endpoints
- UI surfaces
- data stores
- auth roles
- ops obligations
- verification obligations

Panel D — Assumptions and Risks

Two stacked panels:

- assumptions
- risks with severity

Primary operator question answered

“Did the planner actually understand what needed to be built?”

---

9. Repo Inventory Section

Purpose

Show what the Build Agent planned to generate before generation started.

Panels

Panel A — Required Directories

Simple list/tree.

Panel B — Required Files

Table:

- file ID
- path
- category
- trace required
- generated yes/no

Panel C — Optional Files

Table:

- path
- category
- trigger reason
- generated yes/no

Panel D — Proof / Test / Config / Runtime

Grouped sub-panels with counts and paths.

Panel E — Manifest Targets

List of all package-critical artifacts.

Primary operator question answered

“What should have existed in the repo?”

---

10. Traceability Section

Purpose

Show how requirements mapped to build output and proof.

Panels

Panel A — Trace Coverage Summary

Metrics:

- requirement count
- mapped requirement count
- hard requirement count

- mapped hard requirement count
- acceptance with file targets
- acceptance with proof targets

Panel B — Requirement Chain Viewer

Expandable chain:

- requirement
    - work unit
        - acceptance item
            - file target(s)
            - proof target(s)

Panel C — Unmapped Requirements

List:

- requirement ID
- summary
- severity
- likely gap type

Panel D — Trace Gaps

Table:

- gap ID
- gap type
- source ID

- severity
- recommended action

Primary operator question answered

“What requirement disappeared or lost its implementation path?”

---

11. Generation Delta Section

Purpose

Show the difference between planned output and actual output.

Panels

Panel A — Expected vs Produced

Metrics:

- required files planned
- required files produced
- missing required files
- unplanned files produced

Panel B — Missing Required Files

Table:

- file ID
- path

- category
- linked requirement/acceptance
- blocker reason if known

Panel C — Unplanned Generated Files

Table:

- path
- category
- inferred yes/no
- trace linked yes/no

Panel D — Placeholder Violations

Table:

- file path
- linked required unit
- placeholder reason
- severity

Primary operator question answered

“What did the generator miss or fake?”

---

12. Gates Section

Purpose

Show the full gate state and why any gate failed.

Panels

Panel A — Gate Board

Display:

- G-BQ-01 Extraction Completeness
- G-BQ-02 Derived Planning Completeness
- G-BQ-03 Inventory Planning Integrity
- G-BQ-04 Requirement Coverage
- G-BQ-05 Output Sufficiency
- G-BQ-06 Verification Integrity
- G-BQ-07 Packaging Integrity

For each gate show:

- pass/fail
- severity
- short reason
- evidence refs

Panel B — Gate Evidence

When a gate is selected, show:

- source artifacts
- metric values
- blocking issues

- linked failures

Primary operator question answered

“What exact gate stopped this build?”

---

13. Failures Section

Purpose

Show structured failure diagnosis when the build degraded or failed.

Panels

Panel A — Failure Summary

Shows:

- failure class
- retry eligibility
- severity
- top blocker

Panel B — Failing Units

Table:

- unit ID
- unit type
- summary

- severity

Panel C — Missing Artifacts

Table:

- artifact ID
- path
- category
- reason

Panel D — Upstream Blockers

List:

- blocker ID
- source stage
- message

Panel E — Repair Hints

Prioritized list of actions.

Primary operator question answered

“What should be fixed first to recover the build?”

---

14. Packaging Section

Purpose

Show whether the package can be trusted.

Panels

Panel A — Packaging Decision

Shows:

- approved / blocked / failed
- one-line reason

Panel B — Manifest Reconciliation

Metrics:

- manifest targets
- manifest present count
- missing package-critical artifacts
- reconciliation pass/fail

Panel C — Packaging Blockers

List:

- missing trace artifacts
- missing proof artifacts
- gate failures
- manifest mismatches

Primary operator question answered

“Why did packaging pass or why was it blocked?”

---

15. History / Trends Section

Purpose

Show whether build quality is getting better or worse over time.

Panels

Panel A — Run Trend

Track over time:

- extraction coverage
- requirement coverage
- acceptance coverage
- proof completion
- inventory variance
- packaging pass rate

Panel B — Recurring Failure Classes

List or bar chart:

- extraction failure
- planning failure
- inventory failure
- traceability failure
- generation failure
- verification failure

- packaging failure

Panel C — Recurring Missing Artifact Families

Grouped by category:

- infrastructure
- shared contracts
- api
- data
- tests
- proof
- runtime/config

Primary operator question answered

“Is the upgrade actually improving build quality?”

---

16. Right Rail Skeleton

Widgets

Widget 1 — Active Blockers

Top 3–5 current blockers.

Widget 2 — Most Missing Artifacts

Most important missing required files.

Widget 3 — Next Best Fix

Single recommended repair action based on blocker severity and dependency order.

Widget 4 — Quality Delta

Comparison with previous run:

- better
- same
- worse

Widget 5 — Export

Buttons:

- export quality report
- export failure report
- export full evidence bundle

---

17. Empty / Warning / Error States

Empty state

When no run selected:

- show prompt to choose a build run
- show recent runs list

Warning state

When run exists but quality artifacts are partial:

- show partial artifact warning
- identify missing artifact surfaces

Error state

When runtime artifacts are invalid or unreadable:

- show invalid artifact warning
- identify which artifact failed to parse or validate

---

18. Interaction Rules

Filters

Allow filters for:

- file category
- severity
- gate state
- requirement class
- failure class

Drill-down

Every high-level metric should drill into the exact artifacts behind it.

Cross-linking

Clicking a missing file should show:

- linked requirement
- linked acceptance item
- gate impact
- packaging impact

Comparison mode

Recommended:

- compare current run vs previous run
- compare current run vs last successful run

---

19. Recommended Data Wiring

Primary data sources

- `build_quality_report.json`
- `generation_failure_report.json`
- `kit_extraction.json`
- `derived_build_inputs.json`
- `repo_inventory.json`
- `requirement_trace_map.json`

Supporting data

- run manifest
- stage reports
- proof ledger
- packaging manifest

The page should read from emitted runtime artifacts, not ad hoc live calculations wherever possible.

---

20. Page Build Order

First slice

- top bar
- overview
- gates
- failures

Second slice

- extraction
- repo inventory
- generation delta

Third slice

- derived plan

- traceability
- packaging

Fourth slice

- history/trends
- comparison mode
- export bundle

That gives you useful operator value early without waiting for the full page.

---

21. Completion Condition

This page skeleton is complete when:

- every major quality artifact has a clear display area
- the operator can see what was planned, produced, missing, and blocked
- gates and failures are easy to inspect
- packaging trust is visible
- the page supports triage, not just passive reading

REAL DATA WORKSHEET

BUILD_AGENT_QUALITY_UPGRADE — REAL DATA WORKSHEET

0. Worksheet Identity

Doc ID: BAQ-DATA-01
Parent Spec: BAQ-SPEC-01
Parent Page Skeleton: BAQ-PAGE-01
Purpose: define the real data the Build Quality page and enforcement layer must read, where it comes from, how it should be transformed, and what fallback behavior is allowed when the data is partial.

This worksheet is the bridge between the spec and actual implementation.

---

1. Worksheet Goal

The Build Agent quality upgrade only works if the runtime data is real, consistent, and wired from the actual build flow.

This worksheet answers:

- what data each section needs
- which runtime artifact should supply it
- what fields are required
- what transformations are needed
- what to do when data is missing or stale

---

2. Primary Runtime Artifacts

These are the main real-data sources for the update:

1. `kit_extraction.json`
2. `derived_build_inputs.json`
3. `repo_inventory.json`
4. `requirement_trace_map.json`
5. `build_quality_report.json`
6. `generation_failure_report.json`

Supporting artifacts

- run manifest
- stage report(s)
- proof ledger
- packaging manifest
- generated file registry
- verification outputs

---

3. Overview Section — Real Data Mapping

3.1 Final Build Decision Card

Source

- `build_quality_report.json`

Required fields

- `final_decision`
- `status`
- `packaging_eligible`
- `blocking_issues`
- `gate_results`

Transformation

- map `final_decision` to UI verdict label
- use first critical blocking issue as one-line reason
- derive packaging status text from `packaging_eligible`

Fallback

If `build_quality_report.json` missing:

- show `unknown`
- fallback to manifest/stage status only as secondary degraded signal

---

3.2 Coverage Metrics Card

Source

- `build_quality_report.json`

Required fields

- `coverage.extraction_coverage_pct`
- `coverage.requirement_coverage_pct`
- `coverage.acceptance_coverage_pct`
- `coverage.proof_completion_pct`

Transformation

- normalize to percentage display
- clamp invalid values for presentation but flag invalid source data

Fallback

If missing:

- derive partial values from source artifacts only if clearly calculable
- otherwise show unavailable

3.3 Output Integrity Card

Source

- `build_quality_report.json`

Required fields

- `inventory.expected_required_files`
- `inventory.produced_required_files`
- `inventory.missing_required_files`
- `inventory.inventory_variance_pct`

Transformation

- render counts
- derive good/degraded/bad state by policy thresholds

Fallback

- compute from `repo_inventory.json` + generated file registry if needed

3.4 Quality Signals Card

Source

- `build_quality_report.json`

Required fields

- `quality_signals.placeholder_ratio`
- `quality_signals.inferred_artifact_ratio`
- `quality_signals.warning_count`
- `quality_signals.blocking_issue_count`

Transformation

- present as percent/count
- severity color/state comes from thresholds, not arbitrary styling

---

4. Extraction Section — Real Data Mapping

4.1 Section Coverage Table

Source

- `kit_extraction.json`

Required fields

- `source_sections`
- `section_statuses`
- `required_sections`

Join logic

Join `source_sections[].section_id` to `section_statuses[].section_id`

Output columns

- section name
- required
- present

- status
- obligation count
- warning count
- reason

Fallback

If join is incomplete:

- show raw source rows
- mark missing status rows as data integrity issue

4.2 Critical Obligations Panel

Source

- `kit_extraction.json`

Required fields

- `critical_obligations`

Grouping

Group by:

- feature
- data
- api

- security
- ui
- ops
- verification
- packaging

Fallback

No fallback. If absent, show extraction artifact failure.

4.3 Missing / Deferred / Invalid Panel

Source

- `kit_extraction.json`

Required fields

- `missing_sections`
- `deferred_sections`
- `non_applicable_sections`
- `invalid_sections`

Transformation

- combine with `section_statuses.reason` when available

---

5. Derived Plan Section — Real Data Mapping

5.1 Subsystem Map

Source

- `derived_build_inputs.json`

Required fields

- `subsystem_map`
- `feature_map`

Transformation

- aggregate linked features per subsystem
- count required features under each subsystem

5.2 Feature Map

Source

- `derived_build_inputs.json`

Required fields

- `feature_map`

Output columns

- feature name
- subsystem ID
- priority
- required
- linked surfaces count

Linked surfaces derivation

Count matches across:

- `api_surface`
- `ui_surface_map`
- `verification_obligations`

5.3 Surface Summary

Source

- `derived_build_inputs.json`

Required fields

- `api_surface`
- `ui_surface_map`
- `storage_model.stores`
- `auth_model.roles`

- `ops_obligations`
- `verification_obligations`

Transformation

Produce counts and optional drill-down lists

5.4 Assumptions and Risks

Source

- `derived_build_inputs.json`

Required fields

- `assumptions`
- `risks`

---

6. Repo Inventory Section — Real Data Mapping

6.1 Required Directories

Source

- `repo_inventory.json`

Required fields

- `required_directories`

6.2 Required Files Table

Source

- `repo_inventory.json`
- generated file registry

Required fields

- `required_files[].file_id`
- `required_files[].path`
- `required_files[].category`
- `required_files[].trace_required`

Join logic

Join planned required paths or `file_id` against generated registry

Output columns

- file ID
- path
- category
- trace required
- generated yes/no

Fallback

If no generated registry exists:

- show plan-only state
- mark generation reconciliation unavailable

6.3 Optional Files Table

Source

- `repo_inventory.json`
- generated file registry

Required fields

- `optional_files`

Output

- path
- category
- trigger reason
- generated yes/no

6.4 Manifest Targets

Source

- `repo_inventory.json`
- packaging manifest

Required fields

- `manifest_targets`

Join logic

Mark whether each manifest target is present in packaging manifest

---

7. Traceability Section — Real Data Mapping

7.1 Trace Coverage Summary

Source

- `requirement_trace_map.json`

Required fields

- `trace_summary`

Output

- requirement count
- mapped requirement count
- hard requirement count

- mapped hard requirement count
- acceptance with file targets
- acceptance with proof targets

7.2 Requirement Chain Viewer

Source

- `requirement_trace_map.json`

Required fields

- `requirements`
- `work_units`
- `acceptance_items`
- `file_targets`
- `proof_targets`

Transformation

Build nested chain:

- requirement
- linked work units
- linked acceptance items
- linked file targets

- linked proof targets

Data rule

Do not fabricate missing links. Show broken chains explicitly.

7.3 Unmapped Requirements

Source

- `requirement_trace_map.json`

Required fields

- `unmapped_requirements`
- `requirements`

Join logic

Resolve summary text for unmapped IDs from `requirements`

7.4 Trace Gaps

Source

- `requirement_trace_map.json`

Required fields

- `trace_gaps`

---

8. Generation Delta Section — Real Data Mapping

8.1 Expected vs Produced

Source

- `repo_inventory.json`
- generated file registry
- `build_quality_report.json`

Preferred fields

- `build_quality_report.inventory.*`

Fallback calculation

- expected required files = count of `repo_inventory.required_files`
- produced required files = required planned files found in generated registry
- missing required files = expected - produced

8.2 Missing Required Files

Source

- `repo_inventory.json`
- generated file registry

- `requirement_trace_map.json`
- `generation_failure_report.json` if present

Transformation

For each required planned file not found in generated registry:

- attach linked acceptance item if trace exists
- attach blocker reason if available from failure report

8.3 Unplanned Generated Files

Source

- generated file registry
- `repo_inventory.json`

Transformation

Show generated paths not found in required/optional/planned sets

Additional flags

- inferred yes/no
- trace linked yes/no

8.4 Placeholder Violations

Source

- `build_quality_report.json`
- placeholder detection output if separate
- `generation_failure_report.json`

Required fields

Use:

- blocking issues / warnings tagged as placeholder-related
- failing units related to placeholder fulfillment

---

9. Gates Section — Real Data Mapping

9.1 Gate Board

Source

- `build_quality_report.json`

Required fields

- `gate_results`

Output columns

- gate ID
- passed
- severity

- message

Transformation

Map gate IDs to human-readable names:

- G-BQ-01 Extraction Completeness
- G-BQ-02 Derived Planning Completeness
- G-BQ-03 Inventory Planning Integrity
- G-BQ-04 Requirement Coverage
- G-BQ-05 Output Sufficiency
- G-BQ-06 Verification Integrity
- G-BQ-07 Packaging Integrity

9.2 Gate Evidence

Source

- `build_quality_report.json`
- supporting runtime artifacts

Transformation

Each gate detail pane should link to:

- source artifact(s)
- metric values used
- blocking issue entries

- related failure entries

---

10. Failures Section — Real Data Mapping

10.1 Failure Summary

Source

- `generation_failure_report.json`

Required fields

- `failure_class`
- `retry_eligibility`
- `status`
- `upstream_blockers`

Transformation

Top blocker = first highest severity blocker or first blocker entry

---

10.2 Failing Units

Source

- `generation_failure_report.json`

Required fields

- `failing_units`

10.3 Missing Artifacts

Source

- `generation_failure_report.json`

Required fields

- `missing_artifacts`

10.4 Upstream Blockers

Source

- `generation_failure_report.json`

Required fields

- `upstream_blockers`

10.5 Repair Hints

Source

- `generation_failure_report.json`

Required fields

- `repair_hints`

---

11. Packaging Section — Real Data Mapping

11.1 Packaging Decision

Source

- `build_quality_report.json`
- packaging manifest

Required fields

- `packaging_eligible`
- `final_decision`

Transformation

Use build quality report as source of truth.
Packaging manifest is supporting evidence, not primary verdict.

11.2 Manifest Reconciliation

Source

- `repo_inventory.json`
- packaging manifest

- generated file registry

Required fields

- `manifest_targets`

Calculation

- manifest target count
- manifest present count
- missing package-critical artifacts
- reconciliation pass/fail

11.3 Packaging Blockers

Source

- `build_quality_report.json`
- `generation_failure_report.json`

Transformation

Collect blockers tagged:

- packaging
- verification
- trace
- inventory

- manifest mismatch

---

12. History / Trends Section — Real Data Mapping

12.1 Trend Metrics

Source

Historical runs of:

- `build_quality_report.json`
- `generation_failure_report.json`

Required fields per run

- timestamp
- extraction coverage
- requirement coverage
- acceptance coverage
- proof completion
- inventory variance
- final decision
- packaging eligible

Transformation

Build time-series rows per run

12.2 Recurring Failure Classes

Source

Historical `generation_failure_report.json`

Required fields

- `failure_class`

Transformation

Aggregate count by class over chosen time window

12.3 Recurring Missing Artifact Families

Source

Historical `generation_failure_report.json`

- `missing_artifacts[].category` if present
  or category resolved from planned inventory

Transformation

Aggregate missing artifacts by category

---

13. Right Rail — Real Data Mapping

13.1 Active Blockers

Source

- `build_quality_report.json`
- `generation_failure_report.json`

Transformation

Take highest severity items from:

- `blocking_issues`
- `upstream_blockers`

13.2 Most Missing Artifacts

Source

- `generation_failure_report.json`
- missing required file delta

Transformation

Prioritize:

- package-critical
- hard requirement-linked
- proof-critical
- infrastructure-critical

13.3 Next Best Fix

Source

Derived from:

- failure class
- blocker severity
- dependency order
- repair hints

Rule

Recommend the earliest upstream fix with the highest downstream recovery value.

13.4 Quality Delta

Source

Current `build_quality_report.json`
vs previous comparable run

Metrics

- extraction coverage delta
- requirement coverage delta
- acceptance coverage delta
- proof completion delta
- missing required file delta
- decision delta

14. Required Derived Selectors

These selectors should exist so the page does not duplicate raw transform logic everywhere:

- `selectFinalBuildDecision`
- `selectCoverageMetrics`
- `selectOutputIntegrityMetrics`
- `selectExtractionCoverageRows`
- `selectCriticalObligationsByCategory`
- `selectSubsystemSummary`
- `selectSurfaceCounts`
- `selectRequiredFileRows`
- `selectManifestTargetRows`
- `selectTraceCoverageSummary`
- `selectRequirementChains`
- `selectMissingRequiredFiles`
- `selectUnplannedGeneratedFiles`
- `selectGateRows`
- `selectFailureSummary`
- `selectPackagingReconciliation`

- `selectTrendSeries`
- `selectTopBlockers`
- `selectNextBestFix`

---

15. Data Integrity Rules

Rule 1

`build_quality_report.json` is the final verdict source.

Rule 2

If raw artifacts and quality report disagree, show the disagreement as a data integrity warning.

Rule 3

Trace chains must only show actual links present in `requirement_trace_map.json`.

Rule 4

Missing generated file data must not be guessed from packaging output alone.

Rule 5

Trend views must only include runs whose metric artifacts parse successfully.

---

16. Fallback Strategy

Level 1 — Full data available

Use all six quality artifacts plus supporting manifests.

Level 2 — Partial quality artifacts

Use available artifacts and mark missing sections as partial-data state.

Level 3 — Only manifest/stage data available

Show degraded operator mode:

- run status
- package state
- minimal counts
- explicit notice that quality upgrade artifacts are unavailable

Level 4 — Invalid artifact state

Show invalid data warning with artifact names that failed validation.

---

17. Recommended Normalized Frontend View Model

A page-level view model should look roughly like this:

```
type BuildQualityViewModel = {
 runId: string;
 header: {
   buildName?: string;
   buildProfile?: string;
   finalDecision: string;
   packagingEligible: boolean;
   status: string;
   failureClass?: string;
 };
```

```
overview: {
  extractionCoveragePct?: number;
  requirementCoveragePct?: number;
  acceptanceCoveragePct?: number;
  proofCompletionPct?: number;
  expectedRequiredFiles?: number;
  producedRequiredFiles?: number;
  missingRequiredFiles?: number;
  inventoryVariancePct?: number;
  placeholderRatio?: number;
  inferredArtifactRatio?: number;
  warningCount?: number;
  blockingIssueCount?: number;
};
extraction: {
  sectionRows: Array<Record<string, unknown>>;
  criticalObligationGroups: Record<string, Array<Record<string, unknown>>>;
};
derivedPlan: {
  subsystemRows: Array<Record<string, unknown>>;
  featureRows: Array<Record<string, unknown>>;
  surfaceCounts: Record<string, number>;
  assumptions: string[];
  risks: Array<Record<string, unknown>>;
};
inventory: {
  requiredFiles: Array<Record<string, unknown>>;
  optionalFiles: Array<Record<string, unknown>>;
  manifestTargets: Array<Record<string, unknown>>;
};
trace: {
  summary: Record<string, number>;
  chains: Array<Record<string, unknown>>;
  unmapped: Array<Record<string, unknown>>;
  gaps: Array<Record<string, unknown>>;
};
generation: {
  missingRequiredFiles: Array<Record<string, unknown>>;
  unplannedFiles: Array<Record<string, unknown>>;
  placeholderViolations: Array<Record<string, unknown>>;
};
gates: {
  gateRows: Array<Record<string, unknown>>;
};
```

```
 failures?: {
   summary: Record<string, unknown>;
   failingUnits: Array<Record<string, unknown>>;
   missingArtifacts: Array<Record<string, unknown>>;
   blockers: Array<Record<string, unknown>>;
   repairHints: string[];
 };
 packaging: {
   reconciliation: Record<string, unknown>;
   blockers: Array<Record<string, unknown>>;
 };
 trends?: {
   series: Array<Record<string, unknown>>;
   recurringFailureClasses: Array<Record<string, unknown>>;
   recurringMissingFamilies: Array<Record<string, unknown>>;
 };
};
```

---

18. Completion Condition

This worksheet is complete when:

- each page section has a real runtime source
- required fields are identified
- joins and transforms are explicit
- fallback behavior is defined
- the page can be built without inventing data rules ad hoc

INTEGRATION CHECKLIST

BUILD_AGENT_QUALITY_UPGRADE — INTEGRATION CHECKLIST

0. Checklist Identity

Doc ID: BAQ-INT-01
Parent Spec: BAQ-SPEC-01
Parent Worksheet: BAQ-DATA-01
Purpose: provide the implementation integration checklist for wiring the Build Agent quality upgrade into the real system without skipping required dependencies, enforcement points, or packaging protections.

This is the execution control list.

---

1. Use of This Checklist

This checklist should be used in order.

Do not treat it like a loose suggestion list.
Each section is a dependency checkpoint.

A later block should not be marked complete if its upstream requirements are still open.

---

2. Readiness Gate

Confirm before starting implementation

- BAQ-SPEC-01 is accepted as the source-of-truth spec
- BAQ-SCHEMA-01 is accepted as the source-of-truth schema pack
- BAQ-IMP-01 pass order is accepted

- BAQ-HOOK-01 hook order is accepted
- BAQ-PAGE-01 page structure is accepted
- BAQ-DATA-01 real data mapping is accepted
- rollout is being treated as a planned quality upgrade
- failure triage lane exists for bad build-kit sampling

---

3. Codebase Discovery Checklist

Build flow discovery

- identify the Build Agent entry point
- identify the build-kit load step
- identify where authoritative build inputs are assembled
- identify where generation starts
- identify where per-file output is tracked
- identify where verification/proof state is written
- identify where packaging is authorized
- identify where final manifest is built

Artifact discovery

- locate current run manifest contract
- locate current stage report contract

- locate current proof/verification outputs
- locate current generated file registry or equivalent
- locate current packaging manifest contract
- locate current failure/report surfaces

Risk discovery

- identify freeform generation paths that bypass planning
- identify packaging paths that do not read gate state
- identify placeholder-friendly completion logic
- identify weak “file exists only” validation logic
- identify TODO or stubbed tests around build-kit generation

---

4. Schema Integration Checklist

Shared type layer

- create shared enums for status, severity, applicability, gate IDs, failure classes
- create shared runtime artifact base type
- create stable IDs for file targets, requirements, work units, acceptance items, proof targets

Runtime schemas

- add `KIT_EXTRACTION_SCHEMA`

- add `DERIVED_BUILD_INPUTS_SCHEMA`
- add `REPO_INVENTORY_SCHEMA`
- add `REQUIREMENT_TRACE_MAP_SCHEMA`
- add `BUILD_QUALITY_REPORT_SCHEMA`
- add `GENERATION_FAILURE_REPORT_SCHEMA`

Validation layer

- add runtime validation for all six artifacts
- add semantic validation helpers
- add count reconciliation checks
- add trace referential integrity checks
- add manifest reconciliation checks
- add gate-to-final-decision consistency checks

Done condition

- artifacts cannot be emitted in contradictory state
- invalid artifacts surface explicit validation errors

---

5. Extraction Integration Checklist

Wiring

- wire `onBuildAuthorityLoaded`
- wire `onKitExtractionStart`
- wire `onKitExtractionComplete`

Extraction logic

- add authoritative section scanner
- add required section registry
- add section status classifier
- add extraction warning collector
- add critical obligation collector
- add explicit reasons for deferred/not-applicable/invalid states

Enforcement

- missing required sections block forward progress
- silent omission is treated as failure
- `kit_extraction.json` is emitted on every real run
- extraction coverage is computed from required sections only

Done condition

- no build run can continue without a valid extraction artifact

---

6. Derived Inputs Integration Checklist

Wiring

- wire `onDerivedInputsBuild`

Planning surfaces

- add subsystem map builder
- add feature map builder
- add domain model derivation
- add storage model derivation
- add API surface derivation
- add auth model derivation
- add UI surface derivation
- add verification obligations derivation
- add ops obligations derivation
- add assumptions and risks carry-forward

Enforcement

- required features cannot exist without subsystem/build-area mapping
- critical obligations that cannot be normalized block progress
- `derived_build_inputs.json` is emitted before inventory planning

Done condition

- generation no longer depends on summary-only interpretation

---

7. Repo Inventory Integration Checklist

Wiring

- wire `onRepoInventoryPlan`

Inventory logic

- add required directory planner
- add required file planner
- add optional file planner
- add proof file planner
- add test file planner
- add config file planner
- add runtime file planner
- add manifest target planner
- add expected count calculator

Enforcement

- required files resolve to concrete paths
- package-critical targets exist in inventory

- each required file has a stable ID
- vague category-only planning is rejected

Done condition

- every run has a credible planned inventory before generation

---

8. Traceability Integration Checklist

Wiring

- wire `onRequirementTraceBuild`

Trace logic

- add requirement ingestor
- add work-unit linker
- add acceptance linker
- add file target linker
- add proof target linker
- add unmapped requirement collector
- add trace gap collector
- add trace summary calculator

Enforcement

- every hard requirement maps through the chain
- every hard-gate acceptance item has a file target
- proof-required acceptance items have proof targets
- unmapped hard requirements block progress

Done condition

- required implementation obligations can no longer disappear silently

---

9. Sufficiency Integration Checklist

Wiring

- wire `onSufficiencyEvaluation`

Policy logic

- classify required artifact families
- classify optional artifact families
- classify suppressed artifact families
- classify inferred artifact families
- identify blocked artifact families
- add placeholder-risk detection
- add optional-overreach detection

Enforcement

- optional artifacts cannot compensate for missing required artifacts
- placeholder-only fulfillment is rejected for required units
- inferred outputs are explicitly flagged
- no-run state is blocked if minimum sufficient output cannot be established

Done condition

- build output is constrained by sufficiency instead of expansion drift

---

10. Generation Integration Checklist

Wiring

- wire `beforeGenerationStart`
- wire `onFileGenerated`
- wire `onGenerationComplete`

Preflight

- verify extraction artifact exists and passed
- verify derived inputs artifact exists and passed
- verify repo inventory exists and passed
- verify trace artifact exists and passed

- verify sufficiency check passed
- block generation on any unresolved hard blocker

File tracking

- capture generated path for every file
- mark whether generated file was planned
- mark whether generated file is required or optional
- attach trace linkage when available
- flag inferred/unplanned generation
- flag placeholder risk on generation events when detectable

Reconciliation

- compute required produced file count
- compute missing required files
- compute unplanned generated files
- compute inventory variance
- compute placeholder fulfillment violations

Done condition

- generation is trace-backed and measurable against plan

---

11. Gate Integration Checklist

Gate implementation

- implement G-BQ-01 Extraction Completeness
- implement G-BQ-02 Derived Planning Completeness
- implement G-BQ-03 Inventory Planning Integrity
- implement G-BQ-04 Requirement Coverage
- implement G-BQ-05 Output Sufficiency
- implement G-BQ-06 Verification Integrity
- implement G-BQ-07 Packaging Integrity

Evidence rules

- each gate reads from runtime artifacts
- each gate records pass/fail state
- each gate records severity
- each gate records reason/message
- each gate records evidence refs

Enforcement

- hard gate failure blocks packaging
- gate results feed the final quality report
- no green packaging path exists with failed hard gates

Done condition

- build quality is machine-enforced instead of advisory

---

12. Quality Report Integration Checklist

Wiring

- wire `onBuildQualityFinalize`

Report logic

- calculate extraction coverage %
- calculate requirement coverage %
- calculate acceptance coverage %
- calculate proof completion %
- calculate expected/produced/missing required files
- calculate inventory variance %
- calculate placeholder ratio
- calculate inferred artifact ratio
- collect warnings
- collect blocking issues
- resolve packaging eligibility
- resolve final decision

Enforcement

- `build_quality_report.json` always emits on completed, blocked, or failed runs
- final decision is deterministic
- packaging eligibility is false when any hard gate fails
- contradictory quality state is rejected

Done condition

- one authoritative run-quality artifact exists for every run

---

13. Failure Reporting Integration Checklist

Wiring

- wire `onGenerationFailure`

Failure logic

- classify failure into controlled taxonomy
- capture failing units
- capture missing artifacts
- capture upstream blockers
- determine retry eligibility
- generate repair hints
- connect failure artifacts back to planned inventory IDs where possible

Enforcement

- hard-failed or blocked runs emit `generation_failure_report.json`
- missing-artifact failures include concrete paths
- retry eligibility distinguishes safe retry from repair-first cases

Done condition

- degraded runs are diagnosable and trendable

---

14. Verification Integration Checklist

Test coverage

- add extraction coverage tests
- add missing required section tests
- add derived input normalization tests
- add repo inventory correctness tests
- add trace completeness tests
- add placeholder rejection tests
- add expected-vs-produced variance tests
- add hard gate blocking tests
- add packaging rejection tests
- add failure taxonomy tests

Regression cases

- missing infrastructure modules
- missing shared contracts
- missing proof artifacts
- missing verification outputs
- incomplete feature family generation
- manifest/output mismatch
- false-green packaging path

Done condition

- recurring bad build patterns are protected by tests

---

15. Packaging Integration Checklist

Wiring

- wire `beforePackaging`
- wire `onPackagingDecision`

Packaging checks

- reconcile manifest targets against actual outputs
- verify package-critical artifacts exist

- verify all hard gates passed
- verify trace/proof/inventory artifacts exist
- verify packaging authorization reads build quality state directly

Enforcement

- packaging blocks on missing package-critical artifacts
- packaging blocks on manifest mismatch
- packaging blocks on failed hard quality gates
- packaging cannot declare completeness without reconciliation

Done condition

- final package is quality-cleared, not just assembled

---

16. Page / Operator Surface Checklist

Data wiring

- page reads `build_quality_report.json`
- page reads `generation_failure_report.json`
- page reads `kit_extraction.json`
- page reads `derived_build_inputs.json`
- page reads `repo_inventory.json`

- page reads `requirement_trace_map.json`

Selectors

- add final decision selector
- add coverage metric selector
- add extraction rows selector
- add subsystem summary selector
- add required file rows selector
- add trace chain selector
- add missing required files selector
- add gate rows selector
- add failure summary selector
- add packaging reconciliation selector
- add top blockers selector
- add next-best-fix selector

UI states

- add no-run selected state
- add partial-artifact warning state
- add invalid-artifact error state
- add comparison-ready structure for future trend mode

Done condition

- operator can inspect planned, produced, missing, blocked, and packaged state in one surface

---

17. Data Integrity Checklist

Cross-artifact reconciliation

- every trace file target exists in repo inventory
- every missing required file exists in repo inventory
- packaging blockers reconcile with gate state
- final decision reconciles with gate results
- manifest targets reconcile with packaging manifest
- reported counts reconcile with source arrays

Error handling

- contradictory artifacts produce data integrity warnings
- invalid artifacts are not silently swallowed
- partial data state is explicit in the UI and runtime logs

Done condition

- the system fails loudly on contradictory quality state

---

18. Migration / Rollout Checklist

Failure triage first

- collect representative bad build kits
- classify current failures into the new taxonomy
- baseline current coverage and packaging pass behavior

Controlled rollout

- enable extraction artifact first
- enable derived inputs and repo inventory next
- enable traceability next
- enable sufficiency and generation preflight next
- enable gates next
- enable packaging trust boundary next
- enable operator page against real artifacts last or in parallel after artifact stability

Safety controls

- keep rollback path for packaging enforcement
- log all blocked packaging attempts during rollout
- compare old vs new quality decisions during transition if needed

Done condition

- rollout improves quality without losing diagnosability

---

19. Final Acceptance Checklist

Mark this update complete only when all are true:

- Build Agent cannot generate without extraction, derived inputs, inventory, and trace artifacts
- Build Agent cannot package with failed hard quality gates
- required files are planned before generation
- hard requirements are traceable to implementation or proof
- placeholder-only fulfillment is rejected for required units
- every real run emits a final build quality report
- blocked/failed runs emit a failure report
- packaging trust is enforced by reconciliation
- operator surface can explain why a run passed, degraded, or failed
- regression tests protect against recurring omission patterns

---

20. Recommended Immediate Next Build Order

The first execution slice should be:

1. extraction artifact
2. derived inputs artifact

3. repo inventory artifact
4. trace artifact
5. generation preflight
6. hard gates
7. quality report
8. packaging reconciliation

That is the fastest route to higher-quality build kits.

IMPLEMENTATION PROMPT

BUILD_AGENT_QUALITY_UPGRADE — IMPLEMENTATION PROMPT

0. Prompt Identity

Doc ID: BAQ-PROMPT-01
Parent Spec: BAQ-SPEC-01
Purpose: provide the exact implementation prompt to use when building the Build Agent quality upgrade inside the Axion system.

This prompt is meant to keep implementation locked to the approved scope and pass order.

---

1. Primary Implementation Prompt

Use this as the working implementation prompt for the update:

Implement the Build Agent Quality Upgrade for the Axion system.

You are updating the Build Agent so build kits become higher quality, more traceable, and harder to falsely mark as complete.

Follow these requirements exactly.

PRIMARY OBJECTIVE
Raise build-kit quality by enforcing:
1. full build-kit extraction
2. normalized pre-generation planning
3. repo inventory planning before generation
4. requirement-to-file-to-proof traceability
5. minimum sufficient output rules
6. hard quality gates before packaging
7. final quality and failure reporting

DO NOT drift into unrelated UI redesign, control plane redesign, or broad template rewrites.

SYSTEM RULES

- Do not let generation start unless extraction, derived inputs, repo inventory, and requirement trace artifacts are present and valid.
- Do not let packaging complete if any hard build-quality gate fails.
- Do not allow placeholder-only output to satisfy required implementation units.
- Do not silently omit required sections, required files, hard requirements, or proof obligations.
- All quality decisions must read from emitted runtime artifacts, not informal logs or transient memory.

IMPLEMENT THESE RUNTIME ARTIFACTS
1. kit_extraction.json
2. derived_build_inputs.json
3. repo_inventory.json
4. requirement_trace_map.json
5. build_quality_report.json
6. generation_failure_report.json

IMPLEMENT THESE PASSES IN ORDER

PASS 1 — Extraction Foundation
- Add section scanner for authoritative build inputs
- Add required section registry
- Add section status classification:
 consumed | deferred | not_applicable | invalid | missing
- Emit kit_extraction.json
- Fail on silent omission of required sections

PASS 2 — Derived Build Inputs
- Normalize extracted authority into:
 subsystem map
 feature map
 domain model
 storage model
 API surface
 auth model
 UI surface map
 verification obligations
 ops obligations
 assumptions
 risks
- Emit derived_build_inputs.json

PASS 3 — Repo Inventory Planner
- Plan required directories, required files, optional files, proof files, test files, config files, runtime files, and manifest targets

- Required files must be concrete paths with stable IDs
- Emit repo_inventory.json

PASS 4 — Requirement Traceability
- Map requirement -> work unit -> acceptance item -> file target -> proof target
- Record unmapped requirements and trace gaps
- Emit requirement_trace_map.json
- Block progress if hard requirements are unmapped

PASS 5 — Sufficiency Enforcement
- Classify artifact families as required, optional, suppressed, inferred, or blocked
- Reject optional-overreach hiding missing required output
- Reject placeholder-only fulfillment for required units

PASS 6 — Generation Alignment
- Add generation preflight
- Track generated files against planned inventory
- Compute missing required files, unplanned files, inventory variance, placeholder violations

PASS 7 — Hard Gates
Implement these gates:
- G-BQ-01 Extraction Completeness
- G-BQ-02 Derived Planning Completeness
- G-BQ-03 Inventory Planning Integrity
- G-BQ-04 Requirement Coverage
- G-BQ-05 Output Sufficiency
- G-BQ-06 Verification Integrity
- G-BQ-07 Packaging Integrity

PASS 8 — Quality Reporting
- Emit build_quality_report.json
- Include:
 extraction coverage %
 requirement coverage %
 acceptance coverage %
 proof completion %
 expected/produced/missing required files
 inventory variance %
 placeholder ratio
 inferred artifact ratio
 warnings
 blocking issues
 packaging eligibility
 final decision

PASS 9 — Failure Reporting
- Emit generation_failure_report.json on blocked/failed/degraded runs
- Use controlled taxonomy:
 extraction_failure
 planning_failure
 inventory_failure
 traceability_failure
 generation_failure
 placeholder_fulfillment_failure
 verification_failure
 packaging_failure

PASS 10 — Packaging Enforcement
- Reconcile manifest targets with actual outputs
- Read hard gate state directly
- Block packaging on missing package-critical artifacts, failed gates, missing proof, missing trace, or manifest mismatch

HOOKS TO WIRE
- onBuildAuthorityLoaded
- onKitExtractionStart
- onKitExtractionComplete
- onDerivedInputsBuild
- onRepoInventoryPlan
- onRequirementTraceBuild
- onSufficiencyEvaluation
- beforeGenerationStart
- onFileGenerated
- onGenerationComplete
- onVerificationReconcile
- beforePackaging
- onPackagingDecision
- onBuildQualityFinalize
- onGenerationFailure

ENFORCEMENT REQUIREMENTS
- Packaging is a trust boundary.
- A build kit cannot be considered complete unless required extraction, planning, inventory, traceability, generation, verification, and packaging checks all pass.
- All hard gate evidence must come from runtime artifacts.
- Final decision must be deterministic.

UI / PAGE REQUIREMENT

Add or wire a Build Quality operator surface that reads the runtime artifacts and shows:
- overview
- extraction
- derived plan
- repo inventory
- traceability
- generation delta
- gates
- failures
- packaging
- history/trends if available

But do not make UI the first priority. Enforcement spine comes first.

TEST REQUIREMENTS
Add coverage for:
- missing required section
- invalid extraction state
- derived plan normalization
- repo inventory correctness
- trace completeness
- placeholder rejection
- expected vs produced variance
- hard gate blocking
- packaging rejection
- false-green prevention
- failure taxonomy classification

OUTPUT FORMAT
When implementing:
1. identify the existing files/modules to modify
2. list the new files/modules to create
3. implement in pass order
4. explain each code change briefly
5. keep contracts deterministic and typed
6. do not invent extra scope
7. prefer runtime artifact validation over loose heuristics

SUCCESS CONDITION
The Build Agent must not generate or package a build kit unless extraction, derived planning, repo inventory, and requirement traceability are all present and valid.

---

2. Shorter Builder Prompt

Use this version when you want a tighter implementation run:

Implement the Axion Build Agent Quality Upgrade.

Goal:
Make build kits higher quality by adding enforced extraction, normalized planning, repo inventory planning, requirement traceability, sufficiency rules, hard quality gates, and packaging reconciliation.

Required runtime artifacts:
- kit_extraction.json
- derived_build_inputs.json
- repo_inventory.json
- requirement_trace_map.json
- build_quality_report.json
- generation_failure_report.json

Required enforcement:
- generation blocked without extraction + derived inputs + repo inventory + trace
- packaging blocked on failed hard gates
- placeholder-only fulfillment rejected for required units
- hard requirements cannot disappear silently
- required files must be planned before generation

Implement in this order:
1. extraction
2. derived inputs
3. repo inventory
4. traceability
5. sufficiency
6. generation preflight + reconciliation
7. hard gates
8. quality report
9. failure report
10. packaging enforcement

Wire hooks:
onBuildAuthorityLoaded, onKitExtractionStart, onKitExtractionComplete, onDerivedInputsBuild, onRepoInventoryPlan, onRequirementTraceBuild, onSufficiencyEvaluation, beforeGenerationStart, onFileGenerated, onGenerationComplete, onVerificationReconcile, beforePackaging, onPackagingDecision, onBuildQualityFinalize, onGenerationFailure

Add tests for extraction failure, planning failure, missing required files, trace gaps, placeholder rejection, gate blocking, packaging rejection, and false-green prevention.

Do not drift into unrelated redesign work.

---

3. Codebase Audit Prompt

Use this first if you want the system scanned before implementation:

Audit the current Axion Build Agent flow for the Build Agent Quality Upgrade.

Find and report:
1. Build Agent entry point
2. build-kit authority load point
3. generation start point
4. generated file tracking path
5. verification/proof write path
6. packaging authorization path
7. final manifest path
8. current runtime artifacts already emitted
9. places where freeform generation can bypass planning
10. places where packaging does not read gate state
11. weak validation that only checks file existence
12. tests that are TODO, stubbed, or missing for build-kit generation

Then map findings to this upgrade structure:
- extraction
- derived inputs
- repo inventory
- traceability
- sufficiency
- gates
- quality report
- failure report
- packaging reconciliation

Output:
- files to modify
- new files to add
- dependency order
- top risks

- recommended first pass

---

4. Single-Pass Extraction Prompt

Use this if you want to build the first pass only:

Implement only Pass 1 of the Axion Build Agent Quality Upgrade.

Pass 1 scope only:
- authoritative section scanner
- required section registry
- section status classification
- critical obligation collection
- kit_extraction.json emission
- failure on silent omission of required sections

Do not implement later passes yet.

Required status enum:
consumed | deferred | not_applicable | invalid | missing

Output:
1. files changed
2. files created
3. type/schema additions
4. extraction logic added
5. validation logic added
6. how forward progress is blocked on invalid extraction

---

5. Single-Pass Traceability Prompt

Use this for the trace layer only:

Implement only the requirement traceability layer for the Axion Build Agent Quality Upgrade.

Add:
- requirement -> work unit -> acceptance item -> file target -> proof target mapping
- unmapped requirement collection
- trace gap collection

- trace summary calculation
- requirement_trace_map.json emission

Rules:
- every hard requirement must trace forward
- every hard-gate acceptance item must have a file target
- proof-required acceptance items must have proof targets
- unmapped hard requirements must block progress

Output:
- files modified
- files added
- runtime artifact contract
- trace builder logic
- blocking logic
- tests added

---

6. Packaging Enforcement Prompt

Use this when you reach the final enforcement phase:

Implement the packaging trust boundary for the Axion Build Agent Quality Upgrade.

Requirements:
- packaging must read hard gate state from runtime artifacts
- reconcile manifest targets with actual outputs
- block packaging on:
  - failed hard gates
  - missing package-critical artifacts
  - missing trace artifacts
  - missing proof artifacts
  - manifest mismatch
- record packaging decision and reasons
- ensure build_quality_report.json reflects packaging eligibility correctly

Do not redesign packaging UI.
Focus on enforcement and reconciliation logic first.

---

7. Operator Page Prompt

Use this only after the enforcement spine exists:

Build the Build Quality operator page for Axion.

The page must read from:
- build_quality_report.json
- generation_failure_report.json
- kit_extraction.json
- derived_build_inputs.json
- repo_inventory.json
- requirement_trace_map.json

Sections:
- Overview
- Extraction
- Derived Plan
- Repo Inventory
- Traceability
- Generation Delta
- Gates
- Failures
- Packaging
- History / Trends if data exists

Show:
- final decision
- packaging eligibility
- coverage metrics
- missing required files
- unmapped requirements
- gate results
- blockers
- repair hints
- manifest reconciliation

Use emitted runtime artifacts as the source of truth.
Do not invent metrics in the UI layer.

---

8. Anti-Drift Guardrail Add-On

Append this to any implementation prompt when needed:

Anti-drift rules:
- do not add unrelated page redesigns
- do not rename unrelated modules
- do not replace the template system
- do not redesign control planes
- do not introduce speculative abstractions unless required by this upgrade
- do not skip runtime artifacts in favor of log-only solutions
- do not allow packaging to bypass hard gate results
- do not treat placeholder scaffolds as completion for required units

---

9. Best Use Order

Use the prompts in this sequence:

1. **Codebase Audit Prompt**
2. **Primary Implementation Prompt**
3. **Single-Pass prompts** when you want focused implementation slices
4. **Packaging Enforcement Prompt** near the end
5. **Operator Page Prompt** after artifact stability is real

---

10. Completion Condition

This prompt pack is complete when you can hand one of these prompts to a builder and get work that stays aligned with:

- the spec
- the schema pack
- the hook model
- the page skeleton
- the integration checklist

MASTER OUTLINE RECAP

BUILD_AGENT_QUALITY_UPGRADE — MASTER OUTLINE RECAP

0. Recap Identity

Doc ID: BAQ-RECAP-01
Parent Update: Build Agent Quality Upgrade
Purpose: provide one clean master recap of the full update so implementation, planning, and review stay aligned.

This is the consolidated control document.

---

1. Update Name

Build Agent Quality Upgrade

---

2. Core Problem

Your Build Agent is able to produce build kits that look complete but still have bugs, weak output quality, missing implementation depth, missing artifacts, weak traceability, or false-green packaging.

The root issue is that build generation is not yet enforced strongly enough through:

- full build authority extraction
- normalized pre-build planning
- expected repo inventory planning
- requirement-to-output traceability

- sufficiency enforcement
- hard quality gates
- packaging trust checks

---

3. Core Goal

Make the Build Agent produce **higher-quality, more trustworthy build kits** by forcing the system to prove what it consumed, what it planned, what it generated, what it failed to cover, and whether the package is truly complete.

---

4. The Main Rule

The Build Agent must not generate or package a build kit unless extraction, derived planning, repo inventory, and requirement traceability are all present and valid.

That is the backbone of the whole update.

---

5. Main Outcome

After this upgrade, the Build Agent should:

- consume the full authoritative kit
- derive a real build plan before generation
- plan required repo output before generating files
- trace hard requirements to implementation and proof
- reject placeholder-only completion for required units

- block incomplete packaging
- emit quality and failure artifacts that explain what happened

---

6. Main Spec Stack Created

A. Core Spec

BAQ-SPEC-01 — BUILD_AGENT_QUALITY_UPGRADE_SPEC

Defines:

- problem
- purpose
- scope
- flow
- gate model
- artifact contracts
- failure taxonomy
- rollout logic
- success conditions

B. Data Schema Pack

BAQ-SCHEMA-01

Defines:

- `kit_extraction.json`

- `derived_build_inputs.json`
- `repo_inventory.json`
- `requirement_trace_map.json`
- `build_quality_report.json`
- `generation_failure_report.json`

C. Implementation Pass Plan

BAQ-IMP-01

Defines the required pass order:

1. extraction
2. derived inputs
3. repo inventory
4. traceability
5. sufficiency
6. generation alignment
7. hard gates
8. quality report
9. failure report
10. verification expansion
11. packaging reconciliation
12. operator diagnostics

D. Hook Spec

BAQ-HOOK-01

Defines the integration hooks:

- `onBuildAuthorityLoaded`
- `onKitExtractionStart`
- `onKitExtractionComplete`
- `onDerivedInputsBuild`
- `onRepoInventoryPlan`
- `onRequirementTraceBuild`
- `onSufficiencyEvaluation`
- `beforeGenerationStart`
- `onFileGenerated`
- `onGenerationComplete`
- `onVerificationReconcile`
- `beforePackaging`
- `onPackagingDecision`
- `onBuildQualityFinalize`
- `onGenerationFailure`

E. Page Skeleton

BAQ-PAGE-01

Defines the Build Quality operator page structure.

F. Real Data Worksheet

BAQ-DATA-01

Defines what real runtime artifacts power each page section and metric.

G. Integration Checklist

BAQ-INT-01

Defines the execution checklist for wiring the update into the real codebase.

H. Implementation Prompt Pack

BAQ-PROMPT-01

Defines the actual builder prompts to implement the update without drift.

---

7. Core Runtime Artifacts

These are the six key quality artifacts introduced by the update:

1. `kit_extraction.json`

Shows what the Build Agent actually consumed from the build kit.

2. `derived_build_inputs.json`

Shows the normalized planning inputs the Build Agent derived before generation.

3. `repo_inventory.json`

Shows the expected repo/file structure before file generation begins.

4. `requirement_trace_map.json`

Shows how requirements map to work units, acceptance, files, and proof.

5. `build_quality_report.json`

Shows the final quality state, coverage, gate results, and packaging eligibility.

6. `generation_failure_report.json`

Shows why a run degraded, blocked, or failed.

---

8. Main Control Logic

Step 1 — Extraction

The agent must scan and classify all authoritative kit sections.

Step 2 — Derived Planning

The agent must convert extracted authority into normalized planning surfaces.

Step 3 — Repo Inventory

The agent must define required output files and directories before generation.

Step 4 — Traceability

The agent must prove hard requirements have implementation and proof paths.

Step 5 — Sufficiency

The agent must generate only the minimum sufficient artifact set.

Step 6 — Generation Alignment

Actual output must reconcile against the planned inventory.

Step 7 — Verification

Acceptance and proof obligations must reconcile against generated output.

Step 8 — Packaging Trust

Packaging must read hard gate state and reconcile against actual outputs.

---

9. Main Gates

The update introduces seven hard quality gates:

- **G-BQ-01** Extraction Completeness
- **G-BQ-02** Derived Planning Completeness
- **G-BQ-03** Inventory Planning Integrity
- **G-BQ-04** Requirement Coverage
- **G-BQ-05** Output Sufficiency
- **G-BQ-06** Verification Integrity
- **G-BQ-07** Packaging Integrity

If hard gate state fails, packaging must not complete.

---

10. Main Failure Classes

Failed or degraded runs are classified into:

- extraction failure

- planning failure
- inventory failure
- traceability failure
- generation failure
- placeholder fulfillment failure
- verification failure
- packaging failure

This gives you real triage structure instead of vague “build bad” reporting.

---

11. Main Operator Surface

The page this update adds is:

Build Quality

Recommended route:
`/build-quality`

Main sections:

- Overview
- Extraction
- Derived Plan
- Repo Inventory
- Traceability
- Generation Delta
- Gates

- Failures
- Packaging
- History / Trends

This page is meant to explain:

- what the agent planned
- what it produced
- what it missed
- what blocked packaging
- what should be fixed first

---

12. Main Enforcement Rules

These are the most important operating rules from the update:

Rule 1

No silent omission of required kit sections.

Rule 2

No generation without extraction, derived inputs, inventory, and trace.

Rule 3

No hard requirement may disappear without mapping.

Rule 4

No placeholder-only completion for required units.

Rule 5

No packaging with failed hard gates.

Rule 6

No final quality decision based on informal logs only.

Rule 7

Runtime artifacts are the source of truth.

---

13. Main Rollout Strategy

This update should be rolled out in the maintenance posture you already defined as:

- **MM-20 — Failure Triage Mode** first
- then **MM-05 — Planned Upgrade** rollout

That means:

1. collect bad build kits
2. classify current failure patterns
3. baseline current weak spots
4. implement the enforcement spine
5. harden packaging trust
6. add the operator surface and trends

---

14. Best Build Order

The cleanest implementation order remains:

1. extraction artifact
2. derived inputs artifact
3. repo inventory artifact
4. trace artifact
5. sufficiency logic
6. generation preflight and reconciliation
7. hard gates
8. quality report
9. failure report
10. packaging reconciliation
11. verification expansion
12. operator page

That is the fastest path to real quality improvement.

---

15. Immediate Next Recommended Action

The next practical step is not more planning.

It is:

run the codebase audit prompt against the current Build Agent flow
so you can map this full update onto the actual files, modules, and hook points in your system.

That will tell you:

- what files to modify
- what files to create
- where generation begins
- where packaging is authorized
- where current weak validation lives
- what the first real implementation pass should touch

---

16. Final Recap Line

This whole update can be reduced to one sentence:

Turn the Build Agent from a best-effort generator into a traceable, gated, packaging-trusted build system.

CODEBASE AUDIT WORKSHEET

BUILD_AGENT_QUALITY_UPGRADE — CODEBASE AUDIT WORKSHEET

0. Worksheet Identity

Doc ID: BAQ-AUDIT-01
Parent Update: Build Agent Quality Upgrade
Parent Recap: BAQ-RECAP-01
Purpose: provide the audit worksheet used to inspect the current Axion codebase and map the Build Agent Quality Upgrade onto the actual implementation surfaces.

This is the document that turns planning into real file-level work.

---

1. Audit Goal

The audit must identify where the current Build Agent flow already has useful structure, where quality enforcement is missing, and where the new upgrade should attach.

This worksheet answers:

- where build authority enters the system
- where build generation begins
- where generation can bypass planning
- where files are tracked
- where verification is written
- where packaging is authorized
- where manifests are assembled
- where current quality logic is weak

- what files should be modified first

---

2. Audit Output Required

The audit should produce these deliverables:

A. Flow map

A real step-by-step map of the current Build Agent execution path.

B. File/module map

A list of current files/modules that own:

- input loading
- planning
- generation
- verification
- packaging
- reporting

C. Gap map

A list of what the current system does not yet enforce.

D. Injection map

A list of where the new BAQ hooks and artifacts should be wired.

E. First-pass implementation target list

The smallest clean set of files to touch for Pass 1 and Pass 2.

---

3. Audit Method

Step 1 — Find the entry spine

Identify where a build run starts and what function/module acts as the Build Agent entry point.

Step 2 — Follow the authority path

Find where the build kit or build authority bundle is loaded, normalized, or assembled.

Step 3 — Follow the planning path

Find where the system currently decides what to generate.

Step 4 — Follow the generation path

Find where files are created, rendered, written, or staged.

Step 5 — Follow the verification path

Find where gates, tests, or proof outputs are calculated or stored.

Step 6 — Follow the packaging path

Find where manifests and package outputs are assembled and marked complete.

Step 7 — Follow the reporting path

Find where run status, stage reports, and failure reports are emitted.

---

4. Audit Questions

4.1 Entry Point Questions

- what file starts the Build Agent run
- what function/class/method receives the run request
- what context object is passed through the run
- what current stage/order system exists
- does the current flow already have pre-generation checkpoints

4.2 Authority Questions

- where is the build kit loaded
- where are canonical or normalized inputs assembled
- are there multiple input sources merged together
- does the current system distinguish required vs optional inputs
- are missing inputs explicit or silently tolerated

4.3 Planning Questions

- where is the build plan formed
- is there already a work breakdown or acceptance map in runtime use
- is file planning explicit or implicit
- does the generator plan files before writing them
- is there a profile system that affects output shape

4.4 Generation Questions

- what module actually writes or renders files
- can files be generated without trace back to requirements
- is each generated file recorded anywhere
- does the system distinguish planned files vs inferred files
- are placeholders currently treated as acceptable completion

4.5 Verification Questions

- where are gates evaluated
- what evidence do gates read from today
- are there gates that only check file existence
- are proof outputs or verification reports already emitted
- are there false-green paths where completion is declared too early

4.6 Packaging Questions

- where is final packaging triggered
- where is the manifest built
- does packaging read gate state directly
- can packaging complete when required outputs are missing
- is manifest reconciliation strict or loose

4.7 Reporting Questions

- what runtime artifacts already exist

- where are stage reports stored
- where are failure conditions logged or emitted
- is there already a quality summary surface
- what UI or API currently exposes build results

---

5. Audit Sections

Section A — Entry Flow Record

Capture:

- file path
- exported symbol or class
- function name
- input params
- output/result type
- notes on where control branches

Worksheet format

Entry file:
Entry symbol:
Run start function:
Input context:
Output result:
Immediate downstream calls:
Notes:

Section B — Build Authority Record

Capture every file/module that contributes build authority.

For each source record

- file path
- source type
- load function
- fields produced
- required/optional behavior
- current validation behavior
- quality risk

Worksheet format

Authority source:
File:
Loader:
Fields produced:
Required today?:
How missing data is handled:
Observed risk:

Section C — Planning Surface Record

Capture every current planning surface.

Examples of what to look for

- build plan objects
- work breakdown structures
- acceptance maps
- template selection outputs

- repo/file planning logic
- generation config objects

Worksheet format

Planning surface:
File:
Function/module:
What it plans:
Explicit file planning?:
Depends on which upstream artifacts?:
Observed weakness:

Section D — Generation Surface Record

Capture the real file-generation path.

For each generation surface

- file path
- function/module
- generation type
- input artifacts
- output files
- file tracking behavior
- placeholder behavior
- current risk

Worksheet format

Generator module:
File:
Function:
Consumes:

Produces:
Tracks generated files?:
Can generate unplanned files?:
Placeholder risk:
Observed weakness:

Section E — Verification/Gate Record

Capture current gate logic and proof outputs.

For each gate/report surface

- file path
- gate/report name
- pass condition
- evidence source
- output artifact
- weakness

Worksheet format

Gate/report:
File:
Pass condition:
Evidence source:
Output:
Hard block or advisory?:
Observed weakness:

Section F — Packaging Record

Capture final packaging flow.

For each packaging surface

- file path
- function/module
- manifest logic
- packaging authorization logic
- reconciliation behavior
- quality risk

Worksheet format

Packaging module:
File:
Function:
Manifest source:
Authorization source:
Reconciliation strictness:
Observed weakness:

Section G — Existing Artifact Record

List all runtime artifacts already emitted today.

For each artifact

- artifact name
- file path or emitter
- schema/type location
- main fields
- whether it is reliable enough to reuse

Worksheet format

Artifact:
Emitter:

Schema/type location:
Main fields:
Reusable for BAQ?:
Notes:

---

6. Hook Injection Map

This section maps the new BAQ hooks onto the current system.

Required hook mapping table

For each BAQ hook, identify:

- current file to modify
- current function to wrap or extend
- whether new module is needed
- upstream dependency
- downstream dependency
- risk of integration

Hooks to map

- `onBuildAuthorityLoaded`
- `onKitExtractionStart`
- `onKitExtractionComplete`
- `onDerivedInputsBuild`
- `onRepoInventoryPlan`

- `onRequirementTraceBuild`
- `onSufficiencyEvaluation`
- `beforeGenerationStart`
- `onFileGenerated`
- `onGenerationComplete`
- `onVerificationReconcile`
- `beforePackaging`
- `onPackagingDecision`
- `onBuildQualityFinalize`
- `onGenerationFailure`

Worksheet format

Hook:
Current attach point:
File to modify:
Wrapper/extension/new module:
Depends on:
Blocks:
Risk:

---

7. Gap Classification Matrix

Every weakness found during audit should be classified into one of these categories:

- extraction gap
- planning gap

- inventory gap
- traceability gap
- sufficiency gap
- generation gap
- verification gap
- packaging gap
- reporting gap
- test gap

Worksheet format

Gap ID:
Category:
Location:
Description:
Why it matters:
Severity:
Recommended fix pass:

---

8. Risk Flags to Look For

These are the main red flags the audit should hunt for.

High-priority red flags

- build authority is merged informally with no explicit required section registry
- generation starts before file inventory exists
- file generation is prompt-driven without planned file targets
- generated files are not recorded in a structured registry

- gates only check file presence, not credibility or traceability
- packaging trusts stage completion rather than reconciled artifacts
- manifests are built from expected state instead of actual state
- placeholder files count as success for required units
- failure reporting is log-only or too vague to classify
- tests for kit generation are TODO, skipped, or shallow

---

9. Reuse vs Replace Decision Sheet

For each important module discovered, decide:

- reuse as-is
- extend
- wrap
- replace
- deprecate later

Decision rules

Reuse as-is
Only if the module already matches the spec closely.

Extend
If the module is structurally sound but missing fields, checks, or outputs.

Wrap
If the module works but needs a controlled enforcement boundary around it.

Replace
If the module fundamentally works against the new quality model.

Deprecate later
If it is still needed during rollout but should not remain long term.

Worksheet format

Module/file:
Current role:
Decision:
Reason:
Upgrade pass:

---

10. First-Pass Candidate Identification

At the end of the audit, identify the files for the first implementation slice.

Pass 1 candidate set should usually include

- Build Agent run entry file
- build authority loader file
- a new extraction artifact/schema file
- a new extraction service or hook module
- shared status/severity enum/type file
- validation helper file
- test file for extraction

Pass 2 candidate set should usually include

- derived-input compiler file
- derived input schema file
- normalization helpers

- tests for normalization

Worksheet format

Pass:
Files to modify:
Files to create:
Why these first:
Blocked by anything?:

---

11. Audit Summary Template

When the audit is done, the summary should be written in this structure.

A. Current Build Flow Summary

One paragraph describing the actual run path.

B. Strong Existing Pieces

List the current components worth keeping.

C. Main Weaknesses

List the highest-severity quality gaps.

D. Recommended Injection Points

List the best hook points for BAQ.

E. First Implementation Slice

List the first exact files/modules to touch.

F. Main Integration Risks

List the highest-risk coupling or rollout concerns.

---

12. Acceptance Conditions for the Audit

The audit is complete only when all are true:

- the real Build Agent entry point is identified
- the build authority load path is identified
- the generation start point is identified
- the file generation tracking path is identified or confirmed missing
- the verification/gate logic is identified
- the packaging authorization path is identified
- existing runtime artifacts are cataloged
- the BAQ hook injection map is filled out
- major gaps are classified by category
- Pass 1 target files are identified

---

13. Recommended Audit Output Format

The cleanest final audit deliverable should contain:

1. current flow map
2. file/module map
3. gap matrix
4. hook injection table

5. reuse/replace decisions
6. Pass 1 target list
7. Pass 2 target list
8. risk summary

---

14. Immediate Next Document After Audit

Once this worksheet is filled out, the next useful document is:

BUILD_AGENT_QUALITY_UPGRADE — PASS 1 EXECUTION SHEET

That document should convert the audit findings into the exact work plan for:

- extraction hook wiring
- extraction schema/types
- extraction validation
- extraction tests

---

15. Final Line

This worksheet is the point where the update stops being abstract and starts becoming file-level implementation work.

Tab 12